

From Dependent Type Theory to Dependently Typed Higher Order Logic (Technical Report)

Luca Maio¹, Alexander Bentkamp²,
Jasmin Blanchette¹, and Sophie Tourret^{3,4}

¹ Ludwig-Maximilians-Universität München, Munich, Germany
`{luca.maio,jasmin.blanchette}@ifi.lmu.de`

² Cryspen, Paris, France
`alex@cryspen.com`

³ Inria, LORIA, CNRS, Université de Lorraine, Nancy, France
`sophie.tourret@inria.fr`

⁴ Max-Planck-Institut für Informatik, Saarland Informatics Campus,
Saarbrücken, Germany
`sophie.tourret@mpi-inf.mpg.de`

Abstract. There is an expressivity gap between proof assistants based on dependent type theory (DTT), such as Lean and Rocq, and automated theorem provers based on higher order logic (HOL). This gap has been reduced with the introduction of dependently typed HOL (DHOL), along with automated theorem proving for DHOL. To reduce the gap even further, we introduce a translation from a fragment of DTT to polymorphic DHOL. Our translation is sound and lightweight, preserving constructs such as polymorphism and dependent types, and could be used in a hammer system.

1 Introduction

Proof assistants such as Lean [19], Matita [3] and Rocq [5] are based on rich dependent type theories (DTTs) [20], whereas most state-of-the-art automated theorem provers are based on first order logic (FOL) or higher order logic (HOL) [15]. This makes it difficult to develop hammer systems [7] for these proof assistants.

However, the expressivity gap between dependently typed proof assistants and automated provers has shrunk with the emergence of dependently typed HOL (DHOL) [26, 30], along with DHOL specific automated theorem proving techniques. In particular, the Lash [21] system is an extension of the HOL based Satallax [8] tableau prover to DHOL. Moreover, there exists a sound and complete translation from DHOL to HOL [26, 30], which can be used in a hammer.

Nevertheless, there is still a substantial gap between DTT (Sect. 2) and DHOL (Sect. 3). Both are based on λ -calculi and support types that depend on terms. Examples of dependent types include the integers modulo n and the

vectors of length n . A distinguishing feature of traditional DTT is its reliance on the Curry-Howard correspondence: propositions are represented by types, and proofs are represented by terms. Types themselves have types, which are called universes. Even universes can have types, forming a possibly infinite universe hierarchy. DTT is fundamentally intuitionistic but can be extended with classical axioms. The DTTs implemented by Lean, Matita and Rocq are intensional: they use a weak, computational notion of equality for checking types, ensuring that type checking is decidable.

By contrast, DHOL is an extension of classical HOL, which itself is a polymorphic generalisation of Church’s simple theory of types [10]. DHOL has a distinguished type of booleans, and propositions are terms of that type. Types cannot have types, and proofs are not represented within the logic. DHOL is extensional: it uses standard equality for type checking, at the expense of decidability. Due to the lack of higher universes, DHOL is theoretically weaker than DTT with classical axioms, but there are reasons to believe that it is more amenable to automated reasoning.

To bridge the expressivity gap between the two logics and give proof assistant users access to automatic provers, we introduce a translation from a fragment of DTT to DHOL (Sect. 4). The fragment is restricted to three universes and supports only rank 1 (i.e. top level) polymorphism. Although it does not natively support advanced features such as inductive definitions [24] and quotient types [11], it still constitutes a large part of DTT as used in practice. Many DTT formalisations require no more than three universes, higher rank polymorphism is uncommon, and inductive definitions can be axiomatised as a pre-processing step [25].

Our translation is designed to preserve the structure of DTT expressions. In particular, it maps polymorphism to polymorphism and dependent types to dependent types. Based on experience with other translations between logics [6, 13, 18, 25], we hypothesise that such a lightweight translation will lead to more efficient automated proving.

Our translation is sound (Sect. 5). The detailed proofs of soundness and related lemmas can be found in our technical report [17]. We believe that our translation is also complete, but proving this is left for future work.

2 Dependent Type Theory

We describe a fragment of full DTT as presented by Carneiro [9], which is a version of the calculus of inductive constructions. We call it DTT3, or ‘DTT with three universes’. We will later identify a fragment of DTT3 as the domain of our translation.

As a basis, we use a pure type system [4] with

$$\begin{aligned} \text{sorts } \mathcal{S} &= \{U_0, U_1, U_2\} \\ \text{axioms } \mathcal{A} &= \{(U_0 : U_1), (U_1 : U_2)\} \\ \text{rules } \mathcal{R} &= \{(U_i, U_j, U_{\max(i,j)}) \mid 0 \leq i, j \leq 2\} \end{aligned}$$

where imax is defined as follows:

$$\text{imax}(i, j) = \begin{cases} \max(i, j) & \text{if } j > 0 \\ 0 & \text{if } j = 0 \end{cases}$$

We will then add signatures, which are a device to enable global definitions for constants, and an axiomatisation of equality.

Table 1. Abbreviated notations

Abbreviation	Expansion
$\vec{e}$	$e_1, \dots, e_n$
$\vec{e} : \vec{\alpha}$	$e_1 : \alpha_1, \dots, e_n : \alpha_n$
$\vec{e} : \alpha$	$e_1 : \alpha, \dots, e_n : \alpha$
$\forall_{\vec{x}:\vec{\alpha}}$	$\forall_{x_1:\alpha_1} \dots \forall_{x_n:\alpha_n}$
$\lambda_{\vec{x}:\vec{\alpha}}$	$\lambda_{x_1:\alpha_1} \dots \lambda_{x_n:\alpha_n}$

Table 1 summarizes abbreviations that we will use throughout. We use x, y, z to refer to variables, a, b for expressions without variables and c to refer to constants. Sometimes we write $\vec{x}^n$ instead of $\vec{x}$ to indicate the number of components. The abbreviations regarding $\forall$ or λ can include the cases where $n = 0$; a notation such as $\forall_{\vec{x}:\vec{\alpha}} \beta$ then denotes β . Moreover, we write $f \vec{e}$ to denote an iterated curried application.

The syntax of expressions and contexts is given by the following grammar:

$$\begin{aligned} \ell &::= 0 \mid 1 \mid 2 \\ e &::= x \mid c \mid U_\ell \mid \forall_{x:e} e \mid e \ e \mid \lambda_{x:e} e \\ \Gamma &::= \circ \mid \Gamma, x : e \end{aligned}$$

The syntax of signatures $\mathcal{T}$ is given by the following grammar:

$$\mathcal{T} ::= . \mid \mathcal{T}, (c, e) \mid \mathcal{T}, (c, e, e)$$

Signatures formalise global definitions of constants c . Constants can be declared either only with a type α , which we write as (c, α) , or additionally with a value e , which we write as (c, α, e) .

The typing rules of DTT3 follow:

$$\begin{array}{c} \boxed{\Gamma \vdash_{\mathcal{T}} e : \alpha} \\[10pt] \frac{\Gamma \vdash_{\mathcal{T}} \alpha : U_\ell \quad \Gamma \vdash_{\mathcal{T}} e : \beta}{\Gamma, x : \alpha \vdash_{\mathcal{T}} e : \beta} \text{Weak} \quad \frac{\Gamma \vdash_{\mathcal{T}} \alpha : U_\ell}{\Gamma, x : \alpha \vdash_{\mathcal{T}} x : \alpha} \text{Assume} \\[10pt] \frac{}{\circ \vdash_{\mathcal{T}} U_0 : U_1} \text{Univ0} \quad \frac{}{\circ \vdash_{\mathcal{T}} U_1 : U_2} \text{Univ1} \end{array}$$

$$\begin{array}{c}
\frac{\Gamma \vdash_{\mathcal{T}} e_1 : \forall_{x:\alpha} \beta \quad \Gamma \vdash_{\mathcal{T}} e_2 : \alpha}{\Gamma \vdash_{\mathcal{T}} e_1 e_2 : \beta[e_2/x]} \text{Apply} \quad \frac{\Gamma, x : \alpha \vdash_{\mathcal{T}} e : \beta}{\Gamma \vdash_{\mathcal{T}} \lambda_{x:\alpha} e : \forall_{x:\alpha} \beta} \lambda\text{Form} \\
\frac{\Gamma \vdash_{\mathcal{T}} \alpha : \mathbf{U}_{\ell_1} \quad \Gamma, x : \alpha \vdash_{\mathcal{T}} \beta : \mathbf{U}_{\ell_2}}{\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta : \mathbf{U}_{\max(\ell_1, \ell_2)}} \forall\text{Form} \quad \frac{\Gamma \vdash_{\mathcal{T}} e : \alpha \quad \Gamma \vdash_{\mathcal{T}} \alpha \equiv \beta}{\Gamma \vdash_{\mathcal{T}} e : \beta} \text{Conv} \\
\frac{}{\circ \vdash_{\mathcal{T}} c : \mathcal{T}^{\text{ty}}(c)} \text{ConstTy}
\end{array}$$

where $\mathcal{T}^{\text{ty}}(c)$ looks for an entry in $\mathcal{T}$ with c as the first component and returns the second component of the entry if it exists. Later we will also use $\mathcal{T}^{\text{val}}(c)$ to refer to the same lookup but returning the third component if it exists.

We sometimes write $\mathbb{P}$ for $\mathbf{U}_0$. We also sometimes write $\alpha \rightarrow \beta$ for function types instead of $\forall_{x:\alpha} \beta$ when β does not depend on the argument x .

We use $\equiv$ to denote definitional equality [9], including β -, δ - and η -conversion. We also assume proof irrelevance (PI) regarding $\mathbb{P}$, meaning the following holds:

$$\frac{\Gamma \vdash_{\mathcal{T}} p : \mathbb{P} \quad \Gamma \vdash_{\mathcal{T}} h : p \quad \Gamma \vdash_{\mathcal{T}} h' : p}{\Gamma \vdash_{\mathcal{T}} h \equiv h'} \text{PI}$$

By $e_1[e_2/x]$ we denote the capture avoiding substitution of e_2 for x in e_1 . If it is clear which variable is replaced, we simply write $e_1[e_2]$.

We add the following rules for the axiomatisation of propositional, or provable, equality following Angiuli and Gratzner [2]:

$$\begin{array}{c}
\frac{\Gamma \vdash_{\mathcal{T}} \alpha \text{ type} \quad \Gamma \vdash_{\mathcal{T}} a : \alpha \quad \Gamma \vdash_{\mathcal{T}} b : \alpha}{\Gamma \vdash_{\mathcal{T}} a =_{\alpha} b : \mathbb{P}} \text{EqForm} \quad \frac{\Gamma \vdash_{\mathcal{T}} \alpha \text{ type} \quad \Gamma \vdash_{\mathcal{T}} a : \alpha}{\Gamma \vdash_{\mathcal{T}} \text{refl}_a : a =_{\alpha} a} \text{EqIntro} \\
\frac{\Gamma \vdash_{\mathcal{T}} \alpha \text{ type} \quad \Gamma \vdash_{\mathcal{T}} a : \alpha \quad \Gamma \vdash_{\mathcal{T}} b : \alpha \quad \Gamma \vdash_{\mathcal{T}} p : a =_{\alpha} b \quad \Gamma, x : \alpha, y : \alpha, \pi : x =_{\alpha} y \vdash_{\mathcal{T}} Q : \mathbf{U}_{\ell} \quad \Gamma, x : \alpha \vdash_{\mathcal{T}} R : Q[x, x, \text{refl}_x]}{\Gamma \vdash_{\mathcal{T}} \mathcal{J}(R, p) : Q[a, b, p]} \text{EqInd}
\end{array}$$

where $\mathcal{J}(R, \text{refl}_a) \equiv R[a]$ for any $a : \alpha$. Using EqInd, we can then define

$$\text{cast} : \forall_{\alpha, \beta : \mathbf{U}_{\ell}} \forall_{\pi : \alpha =_{\mathbf{U}_{\ell}} \beta} \alpha \rightarrow \beta.$$

We omit the type arguments of **cast** whenever they can be inferred from the context.

We define the following judgements for types, contexts and signatures:

$$\begin{array}{c}
\boxed{\Gamma \vdash \alpha \text{ type}} \quad \frac{\Gamma \vdash_{\mathcal{T}} \alpha : \mathbf{U}_{\ell}}{\Gamma \vdash_{\mathcal{T}} \alpha \text{ type}} \\
\boxed{\vdash_{\mathcal{T}} \Gamma \text{ ok}} \quad \frac{\vdash_{\mathcal{T}} \text{sig}}{\vdash_{\mathcal{T}} \circ \text{ok}} \quad \frac{\vdash_{\mathcal{T}} \Gamma \text{ ok} \quad \Gamma \vdash_{\mathcal{T}} \alpha \text{ type}}{\vdash_{\mathcal{T}} \Gamma, x : \alpha \text{ ok}} \\
\boxed{\vdash_{\mathcal{T}} \text{sig}} \quad \frac{}{\cdot \text{sig}} \quad \frac{\vdash_{\mathcal{T}} \text{sig} \quad \vdash_{\mathcal{T}} \alpha \text{ type}}{\vdash_{\mathcal{T}}, (c, \alpha) \text{ sig}} \quad \frac{\vdash_{\mathcal{T}} \text{sig} \quad \vdash_{\mathcal{T}} \alpha \text{ type} \quad \vdash_{\mathcal{T}} e : \alpha}{\vdash_{\mathcal{T}}, (c, \alpha, e) \text{ sig}}
\end{array}$$

where c must be a fresh name and for technical reasons may not have a name of the form c_{val} for any name c .

3 Dependently Typed Higher Order Logic

We now present our translation's target language, polymorphic DHOL [26], a generalisation of DHOL [30]. Unless otherwise specified, by DHOL we mean the polymorphic variant. DHOL consists of a HOL basis, where the function type $A \rightarrow B$ is replaced by a dependent type $\Pi_{x:A} B$, where x may occur freely in B . Furthermore, there exists a sound and complete translation to HOL [26, 30], which enables automated theorem proving using existing efficient HOL provers.

We present the syntax and rules mainly following Ranalter et al. [26] and add some omitted rules following Rothgang et al. [29, 30]. We also add names to axioms and local assumptions following Rothgang et al. [30]. Finally, we index the $\checkmark$ construct used in the substitutions for formulae by the formula they replace. These changes are needed because we will want the translation function to be injective in the soundness proof.

The syntax of DHOL theories, contexts, substitutions, types and terms is given by the following grammar:

$$\begin{aligned}
\text{theories } T &::= . \mid T, a : \Pi_{\vec{\alpha}} \Pi_{\vec{x}:\vec{A}} : \text{type} \mid T, c : \Pi_{\vec{\alpha}} A \mid T, c \triangleright \Pi_{\vec{\alpha}} F \\
\text{contexts } \Gamma &::= \circ \mid \Gamma, \alpha : \text{type} \mid \Gamma, x : A \mid \Gamma, x \triangleright F \\
\text{substitutions } \gamma &::= \bullet \mid \gamma, A \mid \gamma, t \mid \gamma, \checkmark_F \\
\text{types } A, B &::= a \vec{A} \vec{t} \mid \alpha \mid \Pi_{x:A} B \mid o \\
\text{terms } t, u &::= c \vec{A} \mid x \mid \lambda_{x:A} t \mid t u \mid t \Rightarrow u \mid t =_A u
\end{aligned}$$

The type o denotes the booleans, and terms $F : o$ are also called formulae (or propositions). We call (polymorphic) formulae asserted in theories, written $c \triangleright \Pi_{\vec{\alpha}} F$, global assumptions, and we call (monomorphic) formulae asserted in contexts, written $x \triangleright F$, local assumptions. We use the shorthand notations $\Pi_{\vec{x}:\vec{A}} B$ and $\Pi_{\vec{\alpha}} B$ to mean $\Pi_{x_1:A_1} \dots \Pi_{x_n:A_n} B$ and $\Pi_{\alpha_1} \dots \Pi_{\alpha_n} B$ respectively, and, as in DTT3, $n = 0$ is included in these, in which case they are just B . Theories denote global declarations similar to signatures in DTT3. For substitutions, $\checkmark_F$ is the “term” that may be substituted for a local assumption $x \triangleright F$ if and only if F is provable. Note that **type** is not itself a type. We write $t_1[t_2/x]$ and $t_1[t_2]$ to denote capture avoiding substitution. Again we sometimes write $A \rightarrow B$ instead of $\Pi_{x:A} B$ when B does not depend on x .

Finally, $F \Rightarrow G$ is a dependent implication. This means that to type-check $\Gamma \vdash_T F \Rightarrow G : o$, we need to type-check $\Gamma \vdash_T F : o$ as usual but may assume F as true when type checking G , so we need to prove $\Gamma, x \triangleright F \vdash_T G : o$. This makes it possible to state formulae such as $\Gamma \vdash a =_A b \Rightarrow f a =_{B[a]} f b$ for some type A , a type family B and a dependent function $f : \Pi_{x:A} B$, since the truth of $a =_A b$ may be assumed during type checking. DHOL being an extensional logic, it has only one kind of equality instead of distinguishing between definitional and propositional equality as is done in DTT3. As a result, under the assumption that $a =_A b$, the types $B[a]$ and $B[b]$ can be unified directly, without needing to add constructs such as DTT3's **cast**. On the other hand, this means that type checking in DHOL is undecidable. This is considered

tolerable because the primary use case of DHOL, automated theorem proving, is undecidable regardless.

We define the remaining logical connectives and quantifiers following Andrews [1] and Rothgang et al. [30]:

$$\begin{aligned}
\top &:= (\lambda_{x:o} x) =_{o \rightarrow o} (\lambda_{x:o} x) \\
\perp &:= (\lambda_{x:o} x) =_{o \rightarrow o} (\lambda_{x:o} \top) \\
\neg F &:= F =_o \perp \\
\forall_{x:A} F &:= \lambda_{x:A} F =_{A \rightarrow o} \lambda_{x:A} \top \\
\exists_{x:A} F &:= \neg \forall_{x:A} \neg F \\
F \wedge G &:= \neg (F \Rightarrow \neg G) \\
F \vee G &:= \neg F \Rightarrow G
\end{aligned}$$

where F and G are formulae.

The inference rules for well formed theories T are as follows:

$$\begin{array}{c}
\boxed{\vdash T \text{ Thy}} \quad \frac{}{\vdash \cdot \text{ Thy}} \text{ThyEmpty} \quad \frac{\vdash T \text{ Thy} \quad a \notin T \quad \vdash_T \Delta \text{ Ctx} \quad \Delta = \vec{\alpha} : \text{type}, \vec{x} : \vec{A}}{\vdash T, a : \Pi_{\Delta} : \text{type Thy}} \text{ThyTp} \\
\\
\frac{\vdash T \text{ Thy} \quad c \notin T \quad \Delta \vdash_T A : \text{type} \quad \Delta = \vec{\alpha} : \text{type}}{\vdash T, c : \Pi_{\Delta} A \text{ Thy}} \text{ThyCon} \\
\\
\frac{\vdash T \text{ Thy} \quad c \notin T \quad \Delta \vdash_T F : o \quad \Delta = \vec{\alpha} : \text{type}}{\vdash T, c \triangleright \Pi_{\Delta} F \text{ Thy}} \text{ThyAsn}
\end{array}$$

The rules for contexts are given on the left hand side below, and those for substitutions for them are given on the right hand side:

$$\begin{array}{cc}
\boxed{\vdash_T \Delta \text{ Ctx}} & \boxed{\Gamma \vdash_T \Delta \leftarrow \delta} \\
\\
\frac{\vdash T \text{ Thy}}{\vdash_T \circ \text{ Ctx}} \text{CtxEmpty} & \frac{\vdash_T \Gamma \text{ Ctx}}{\Gamma \vdash_T \circ \leftarrow \bullet} \text{SubEmpty} \\
\\
\frac{\vdash_T \Delta \text{ Ctx}}{\vdash_T \Delta, \alpha : \text{type Ctx}} \text{CtxTp} & \frac{\Gamma \vdash_T \Delta \leftarrow \delta \quad \Gamma \vdash_T A : \text{type}}{\Gamma \vdash_T \Delta, \alpha : \text{type} \leftarrow \delta, A} \text{SubTp} \\
\\
\frac{\vdash_T \Delta \text{ Ctx} \quad \Delta \vdash_T A : \text{type}}{\vdash_T \Delta, x : A \text{ Ctx}} \text{CtxVar} & \frac{\Gamma \vdash_T \Delta \leftarrow \delta \quad \Delta \vdash_T A : \text{type} \quad \Gamma \vdash_T t : A[\delta]}{\Gamma \vdash_T \Delta, x : A \leftarrow \delta, t} \text{SubVar} \\
\\
\frac{\vdash_T \Delta \text{ Ctx} \quad \Delta \vdash_T F : o}{\vdash_T \Delta, x \triangleright F \text{ Ctx}} \text{CtxAsn} & \frac{\Gamma \vdash_T \Delta \leftarrow \delta \quad \Delta \vdash_T F : o \quad \Gamma \vdash_T F[\delta]}{\Gamma \vdash_T \Delta, x \triangleright F \leftarrow \delta, \checkmark_F} \text{SubAsn}
\end{array}$$

The rules for looking up a type, term or assumption in a theory are given on the left hand side, and the corresponding rules for lookup in a context are given on the right hand side:

$$\begin{array}{c}
\boxed{\Gamma \vdash_T A : \text{type}} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad \Gamma \vdash_T \Delta \leftarrow \delta \quad a : \Pi_{\Delta} : \text{type in } T}{\Gamma \vdash_T a \delta : \text{type}} \text{TpSym} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad \alpha : \text{type in } \Gamma}{\Gamma \vdash_T \alpha : \text{type}} \text{TpVar} \\
\\
\boxed{\Gamma \vdash_T t : A} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad \Gamma \vdash_T \Delta \leftarrow \delta \quad c : \Pi_{\Delta} B \text{ in } T}{\Gamma \vdash_T c \delta : B[\delta]} \text{TermSym} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad x : A \text{ in } \Gamma}{\Gamma \vdash_T x : A} \text{TermVar} \\
\\
\boxed{\Gamma \vdash_T F} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad \Gamma \vdash_T \Delta \leftarrow \delta \quad c \triangleright \Pi_{\Delta} F \text{ in } T}{\Gamma \vdash_T F[\delta]} \text{ValidSym} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad x \triangleright F \text{ in } \Gamma}{\Gamma \vdash_T F} \text{ValidVar}
\end{array}$$

The rules for the formation and the terms of booleans are as follows:

$$\begin{array}{c}
\frac{\vdash_T \Gamma \text{Ctx}}{\Gamma \vdash_T o : \text{type}} \text{TpBool} \quad \frac{\Gamma \vdash_T t : A \quad \Gamma \vdash_T u : A}{\Gamma \vdash_T t =_A u : o} \text{TermEq} \\
\\
\frac{\Gamma \vdash_T F : o \quad \Gamma, x \triangleright F \vdash_T G : o}{\Gamma \vdash_T F \Rightarrow G : o} \text{TermImpl}
\end{array}$$

Type formation, λ -abstraction and application are defined as follows:

$$\begin{array}{c}
\frac{\Gamma \vdash_T A : \text{type} \quad \Gamma, x : A \vdash_T B : \text{type}}{\Gamma \vdash_T \Pi_{x:A} B : \text{type}} \text{TpPi} \quad \frac{\Gamma \vdash_T A : \text{type} \quad \Gamma, x : A \vdash_T t : B}{\Gamma \vdash_T \lambda_{x:A} t : \Pi_{x:A} B} \text{TermLambda} \\
\\
\frac{\Gamma \vdash_T t : \Pi_{x:A} B \quad \Gamma \vdash_T u : A}{\Gamma \vdash_T t u : B[u]} \text{TermApply}
\end{array}$$

The type equality judgement is defined by the following rules:

$$\begin{array}{c}
\boxed{\Gamma \vdash_T A \equiv A'} \quad \frac{\vdash_T \Gamma \text{Ctx} \quad \alpha : \text{type in } \Gamma}{\Gamma \vdash_T \alpha \equiv \alpha} \text{TpEqVar} \\
\\
\frac{\vdash_T \Gamma \text{Ctx} \quad \Gamma \vdash_T \delta \equiv_{\Delta} \delta' \quad a : \Pi_{\Delta} \text{type in } T}{\Gamma \vdash_T a \delta \equiv a \delta'} \text{TpEqSym} \quad \frac{\vdash_T \Gamma \text{Ctx}}{\Gamma \vdash_T o \equiv o} \text{TpEqBool} \\
\\
\frac{\Gamma \vdash_T A \equiv A' \quad \Gamma, x : A \vdash_T B \equiv B'}{\Gamma \vdash_T \Pi_{x:A} B \equiv \Pi_{x:A'} B'} \text{TpEqPi} \quad \frac{\Gamma \vdash_T t : A \quad \Gamma \vdash_T A \equiv A'}{\Gamma \vdash_T t : A'} \text{TermConvert}
\end{array}$$

where $\Gamma \vdash_T \delta \equiv_{\Delta} \delta'$ abbreviates the component-wise provable equality of two substitutions for Δ .

The following rules were omitted by Ranalter et al. [26], so we adapt them from Rothgang et al. [29, 30]:

$$\begin{array}{c}
\frac{\Gamma \vdash_T \lambda_{x:A} t : \Pi_{x:A} B \quad \Gamma \vdash_T u : A}{\Gamma \vdash_T (\lambda_{x:A} t) u =_{B[u]} t[u]} \text{TermBeta} \qquad \frac{\Gamma \vdash_T t : \Pi_{x:A} B \quad x \text{ not in } \Gamma}{\Gamma \vdash_T t =_{\Pi_{x:A} B} \lambda_{x:A} t x} \text{TermEta} \\
\\
\frac{\Gamma \vdash_T A \equiv A' \quad \Gamma, x : A \vdash_T t =_B t'}{\Gamma \vdash_T \lambda_{x:A} t =_{\Pi_{x:A} B} \lambda_{x:A'} t'} \text{CongLambda} \qquad \frac{\Gamma \vdash_T t =_A t' \quad \Gamma \vdash_T f =_{\Pi_{x:A} B} f'}{\Gamma \vdash_T f t =_{B[t]} f' t'} \text{CongApply} \\
\\
\frac{\Gamma \vdash_T t : A}{\Gamma \vdash_T t =_A t} \text{Refl} \quad \frac{\Gamma \vdash_T t =_A s}{\Gamma \vdash_T s =_A t} \text{Sym} \quad \frac{\Gamma \vdash_T F =_o F' \quad \Gamma \vdash_T F'}{\Gamma \vdash_T F} \text{CongValid} \\
\\
\frac{\Gamma, x \triangleright F \vdash_T G}{\Gamma \vdash_T F \Rightarrow G} \text{ImplIntro} \qquad \frac{\Gamma \vdash_T F \Rightarrow G \quad \Gamma \vdash_T F}{\Gamma \vdash_T G} \text{MP}
\end{array}$$

Finally, it is possible to add the following rule for boolean extensionality:

$$\frac{\Gamma \vdash_T t : \Pi_{x:o} o \quad \Gamma \vdash_T t \perp \quad \Gamma \vdash_T t \top}{\Gamma, x : o \vdash_T t x} \text{BoolExt}$$

If we add this rule, then a suitable rule for the law of the excluded middle, as well as suitable axiomatisations of the empty type, the singleton type and the co-product, are needed in DTT3 to prove the our translation's soundness.

The main differences between DHOL and DTT3 can be seen even more clearly when looking at an alternative presentation of DHOL, also as an extension of a pure type system. DTT3 features nine possible combinations for the rule $\forall\text{Form}$. HOL can be viewed as the pure type system with sorts $\{o, \text{type}, \text{kind}\}$, axioms $\{(o : \text{type}), (\text{type} : \text{kind})\}$ and rules $\{(o, o, o), (\text{type}, o, o), (\text{type}, \text{type}, \text{type})\}$ [14]. DHOL then allows for formations $(\text{type}, \text{kind}, \text{kind})$ in theories and adds the three rules for polymorphism, restricted to rank 1 and also to theories. This then leaves the rules $(o, \text{type}, \text{type})$ and $(o, \text{kind}, \text{kind})$, which DHOL does not support. This would correspond to abstraction over proof obligations or part of functions with pre-conditions.

Full DTT is further strictly stronger than DHOL. This is due to the former's infinite universe hierarchy, which allows for the same strength as set theory with inaccessible cardinals [32]. By contrast, DHOL has the same strength as HOL by the existence of a sound and complete translation. HOL, in turn, is subsumed by the calculus of constructions, which can be expressed in set theory without additional cardinals [32].

We believe that DHOL lends itself well to automated theorem proving, due to its extensionality and because it resembles HOL. This might enable a term rewriting based proof strategy such as superposition, which is highly successful in FOL and HOL, to be implemented in DHOL and achieve high success rates.

4 The Translation

Here we present our translation from DTT3 to DHOL. The main difficulty in defining it is to map the expressions of DTT3 to either DHOL types or terms depending on the situation. We achieve this through the definition of several mutually recursive translation functions. Another important aspect is the move from intensional to extensional equality handling. Because of this we strip all casts from expressions, since they are not needed in the extensional setting. However, due to type checking DTT3 expressions being necessary in some cases of our translation, we can not just drop all of them immediately, since they might be necessary at some point, therefore we had to integrate the dropping of casts with the translation.

Definition 1. Let Γ be a DTT3 context and $\mathcal{T}$ be a DTT3 signature. Then:

- $\llbracket \cdot \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}}$ maps DTT3 expressions to DHOL types or terms,
- $\llbracket \cdot \rrbracket_{\text{ty}}^{\Gamma; \mathcal{T}}$ maps DTT3 expressions α where $\Gamma \vdash_{\mathcal{T}} \alpha : \forall \vec{x} : \vec{\alpha} \mathbf{U}_1$ to DHOL types,
- $\llbracket \cdot \rrbracket_{\text{ter}}^{\Gamma; \mathcal{T}}$ maps DTT3 expressions e where $\Gamma \vdash_{\mathcal{T}} e : \forall \vec{x} : \vec{\beta} \alpha$ and $\Gamma \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$ to DHOL terms,
- $\llbracket \cdot \rrbracket_{\text{sig}}^{\mathcal{T}}$ maps DTT3 signatures to DHOL theories and
- $\llbracket \cdot \rrbracket_{\text{ctx}}^{\mathcal{T}}$ maps DTT3 contexts to DHOL contexts.

Moreover, we introduce the function `rmc`, which takes a DTT3 expression and if it is of the form `cast p e` it replaces it by e . Finally, we let $\llbracket \cdot \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}} = \llbracket \cdot \rrbracket_{\text{ty}}^{\Gamma; \mathcal{T}} \circ \text{rmc}$ and $\llbracket \cdot \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} = \llbracket \cdot \rrbracket_{\text{ter}}^{\Gamma; \mathcal{T}} \circ \text{rmc}$.

$$\llbracket \Gamma \rrbracket_{\text{ctx}}^{\mathcal{T}} = \begin{cases} \circ & \text{if } \Gamma = \circ \\ \llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}}, x : \text{type} & \text{if } \Gamma = \Gamma', x : \mathbf{U}_1 \\ \llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}}, x : o & \text{if } \Gamma = \Gamma', x : \mathbb{P} \\ \llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}}, x \triangleright \llbracket e \rrbracket_{\text{ter}}^{\Gamma'; \mathcal{T}} & \text{if } \Gamma = \Gamma', x : e \text{ with } e : \mathbb{P} \\ \llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}}, x : \llbracket e \rrbracket_{\text{ty}}^{\Gamma'; \mathcal{T}} & \text{if } \Gamma = \Gamma', x : e \text{ with } e \neq \mathbb{P} \text{ and } e : \mathbf{U}_1 \end{cases}$$

$$\llbracket \mathcal{T} \rrbracket_{\text{sig}} = \begin{cases} . & \text{if } \mathcal{T} = . \\ \llbracket \mathcal{T}', (c, \alpha) \rrbracket_{\text{sig}}, c_{\text{val}} \triangleright c = \llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma; \mathcal{T}} \llbracket e \rrbracket_{\text{ter}}^{\Gamma; \mathcal{T}} & \text{if } \mathcal{T} = \mathcal{T}', (c, \alpha, e) \\ \llbracket \mathcal{T}' \rrbracket_{\text{sig}}, c \triangleright \Pi_{\vec{x}^n} \llbracket e \rrbracket_{\text{ter}}^{\Gamma, \vec{x}^n; \mathbf{U}_1; \mathcal{T}} & \text{if } \mathcal{T} = \mathcal{T}', (c, \forall \vec{x}^n; \mathbf{U}_1 e) \\ & \text{with } e : \mathbb{P} \\ \llbracket \mathcal{T}' \rrbracket_{\text{sig}}, c : \Pi_{\vec{x}^n} \Pi_{\Delta} : \text{type} & \\ \text{if } \mathcal{T} = \mathcal{T}', (c, \forall \vec{x}^n; \mathbf{U}_1 \forall \vec{y}^m; \vec{\alpha}^m \mathbf{U}_1) & \\ \llbracket \mathcal{T}' \rrbracket_{\text{sig}}, c : \Pi_{\vec{x}^n} \llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma, \vec{x}^n; \mathbf{U}_1; \mathcal{T}} & \text{if } \mathcal{T} = \mathcal{T}', (c, \forall \vec{x}^n; \mathbf{U}_1 \alpha) \\ & \text{with } \alpha : \mathbf{U}_1 \end{cases}$$

where $\Delta = y_1 : \llbracket \alpha_1 \rrbracket_{\text{ty}}^{\Gamma, \vec{x}^n; \mathbf{U}_1; \mathcal{T}}, y_2 : \llbracket \alpha_2 \rrbracket_{\text{ty}}^{\Gamma, \vec{x}^n; \mathbf{U}_1, y_1 : \alpha_1; \mathcal{T}}, \dots,$
 $y_m : \llbracket \alpha_m \rrbracket_{\text{ty}}^{\Gamma, \vec{x}^n; \mathbf{U}_1, \vec{y}^{m-1}; \vec{\alpha}^{m-1}; \mathcal{T}}$

$$\begin{aligned}
\llbracket e \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}} &= \begin{cases} \llbracket e_1 \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}} \llbracket e_2 \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}} & \text{if } e = e_1 \ e_2 \text{ and } e_1 \neq \text{cast } p \\ & \text{and not } \Gamma \vdash_{\mathcal{T}} e : \alpha \text{ with } \Gamma \vdash_{\mathcal{T}} \alpha : \mathbb{P} \\ \llbracket e \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}} & \text{if } \Gamma \vdash_{\mathcal{T}} e : \forall_{\vec{x}} : \vec{\alpha} \mathbf{U}_1 \\ & \text{and } e = \text{cast } p \ e_1 \text{ or } e \text{ is not an application} \\ \llbracket e \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} & \text{if } \Gamma \vdash_{\mathcal{T}} e : \forall_{\vec{x}} : \vec{\beta} \alpha \text{ with } \Gamma \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1 \\ & \text{and } e = \text{cast } p \ e_1 \text{ or } e \text{ is not an application} \\ \checkmark_{\llbracket \alpha \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}}} & \text{if } \Gamma \vdash_{\mathcal{T}} e : \alpha \text{ with } \Gamma \vdash_{\mathcal{T}} \alpha : \mathbb{P} \end{cases} \\
\llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma; \mathcal{T}} &= \begin{cases} c & \text{if } \alpha = c \text{ is a constant} \\ x & \text{if } \alpha = x \text{ is a variable} \\ o & \text{if } \alpha = \mathbb{P} \\ \llbracket \alpha \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}} & \text{if } \alpha = \alpha_1 \ \alpha_2 \\ \prod_{x: \llbracket \alpha_1 \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}}} \llbracket \alpha_2 \rrbracket_{\text{ty}0}^{\Gamma, x: \alpha_1; \mathcal{T}} & \text{if } \alpha = \forall_{x: \alpha_1} \alpha_2 \end{cases} \\
\llbracket e \rrbracket_{\text{ter}}^{\Gamma; \mathcal{T}} &= \begin{cases} c & \text{if } e = c \text{ is a constant} \\ x & \text{if } e = x \text{ is a variable} \\ \lambda_{x: \llbracket \alpha \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}}} . \llbracket e_1 \rrbracket_{\text{exp}}^{\Gamma, x: \alpha; \mathcal{T}} & \text{if } e = \lambda_{x: \alpha} . e_1 \\ \llbracket e \rrbracket_{\text{exp}}^{\Gamma; \mathcal{T}} & \text{if } e = e_1 \ e_2 \\ \top & \text{if } e = \top \\ \perp & \text{if } e = \perp \\ \llbracket e_1 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} \diamond \llbracket e_2 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} & \text{if } e = e_1 \diamond e_2 \text{ with } \diamond \in \{\wedge, \vee\} \\ \neg \llbracket e_1 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} & \text{if } e = \neg e_1 \\ \exists_{x: \llbracket \alpha \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}}} \llbracket e_1 \rrbracket_{\text{ter}0}^{\Gamma, x: \alpha; \mathcal{T}} & \text{if } e = \exists_{x: \alpha} e_1 \\ \forall_{x: \llbracket \alpha \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}}} \llbracket e_1 \rrbracket_{\text{ter}0}^{\Gamma, x: \alpha; \mathcal{T}} & \text{if } e = \forall_{x: \alpha} e_1 \text{ with } \Gamma \vdash_{\mathcal{T}} \alpha : \beta \text{ and } \beta \neq \mathbb{P} \\ \llbracket \alpha \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} \Rightarrow \llbracket e_2 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} & \text{if } e = \forall_{x: \alpha} e_1 \text{ with } \Gamma \vdash_{\mathcal{T}} \alpha : \mathbb{P} \\ \llbracket e_1 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} =_{\llbracket \alpha \rrbracket_{\text{ty}0}^{\Gamma; \mathcal{T}}} \llbracket e_2 \rrbracket_{\text{ter}0}^{\Gamma; \mathcal{T}} & \text{if } e = (e_1 =_{\alpha} e_2) \end{cases}
\end{aligned}$$

The annotations on some of the translation functions with a context and signature are necessary for the definition, but we omit them when they can easily be inferred.

The translation functions for DTT3 expressions inspect them without β -, δ -, or η -normalisation. The function definition mapping to DHOL terms includes cases for $\top$, $\perp$, $\wedge$, $\vee$, $\neg$ and $\exists$, which are not built into DTT3, and map these to the corresponding DHOL constructs. Our hope is that this will facilitate more efficient theorem proving on the resulting DHOL formulae.

The split into multiple translation functions is necessary in order to ensure that certain expressions get translated to DHOL terms and others to DHOL types. This is especially relevant for dependent function types, which may be viewed as either (dependent) implications, meaning terms, or as said function types in DHOL, depending on the context.

Due to the differences between DTT3 and DHOL we have pointed out at the end of Sect. 3, we also restrict the translation's domain. We define the *translatable*

fragment by, excluding higher rank polymorphism, enforcing all constants to be fully applied, and disallowing λ -abstractions over propositions, as well as the corresponding function types. The last restriction is due to DHOL not supporting abstraction over proof obligations.

Definition 2 (Translatability). Given a DTT3 signature $\mathcal{T}$ and a DTT3 context Γ , we define $(\mathcal{T}, \Gamma)$ -pre-translatability of a DTT3 expression e inductively:

1. variables, constants and $\mathbb{P}$ are $(\mathcal{T}, \Gamma)$ -pre-translatable.
2. $e_1 \ e_2$ is $(\mathcal{T}, \Gamma)$ -pre-translatable, if e_1 and e_2 are.
3. $e_1 =_\alpha e_2$ is $(\mathcal{T}, \Gamma)$ -pre-translatable, if e_1, e_2 and α are.
4. $\lambda_{x:\alpha} e_1$ is $(\mathcal{T}, \Gamma)$ -pre-translatable, if
 - (a) α is $(\mathcal{T}, \Gamma)$ -pre-translatable,
 - (b) e_1 is $(\mathcal{T}, \Gamma, x : \alpha)$ -pre-translatable,
 - (c) $\Gamma \vdash_{\mathcal{T}} \lambda_{x:\alpha} e_1 : \forall_{x:\alpha} \beta$ with $\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta \not\vdash \mathbb{U}_2$,
 - (d) and $\Gamma \vdash_{\mathcal{T}} \alpha \not\vdash \mathbb{P}$.
5. $\forall_{x:\alpha} \beta$ is $(\mathcal{T}, \Gamma)$ -pre-translatable, if
 - (a) α is $(\mathcal{T}, \Gamma)$ -pre-translatable,
 - (b) β is $(\mathcal{T}, \Gamma, x : \alpha)$ -pre-translatable,
 - (c) $\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta \not\vdash \mathbb{U}_2$,
 - (d) and if $\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta \not\vdash \mathbb{P}$, then $\Gamma \vdash_{\mathcal{T}} \alpha \not\vdash \mathbb{P}$.

We call e $(\mathcal{T}, \Gamma)$ -translatable if it is $(\mathcal{T}, \Gamma)$ -pre-translatable and all occurring constants are fully applied, meaning the type of all subexpressions $c \ \vec{e}$, where $\vec{e}$ are all arguments applied to c , is not a function type.

We define *translatability* of a DTT3 signature $\mathcal{T}$ inductively:

1. $\cdot$ is translatable,
2. $\mathcal{T}, (c, \alpha)$ is translatable, if $\mathcal{T}$ is translatable and α is $(\mathcal{T}, \circ)$ -translatable or if α is of the form $\forall_{\vec{x}:\mathbb{U}_1} \forall_{\vec{y}:\vec{\alpha}} \beta$, all α_i are $(\mathcal{T}, \vec{x} : \mathbb{U}_1, \vec{y}^{i-1} : \vec{\alpha}^{i-1})$ -translatable and β is $(\mathcal{T}, \vec{x} : \mathbb{U}_1, \vec{y} : \vec{\alpha})$ -translatable or $\beta = \mathbb{U}_1$,
3. $\mathcal{T}, (c, \alpha, e)$ is translatable, if $\mathcal{T}, (c, \alpha)$ is translatable and e is $(\mathcal{T}, \circ)$ -translatable.

Given a DTT3 signature $\mathcal{T}$, we define $\mathcal{T}$ -translatability of a DTT3 context Γ inductively:

1. $\circ$ is $\mathcal{T}$ -translatable,
2. $\Gamma, x : \alpha$ is $\mathcal{T}$ -translatable, if Γ is $\mathcal{T}$ -translatable and either α is $(\mathcal{T}, \Gamma)$ -translatable or $\alpha = \mathbb{U}_1$.

We omit $\mathcal{T}$ and Γ if they are clear from the context.

Theorem 1 (Well definedness). *The translation functions from Def. 1 are well defined on the translatable fragment of DTT3, meaning they are defined for all well typed translatable arguments.*

Proof. The proof is by mutual induction on the definition of the translation functions.

The cases for $\llbracket \cdot \rrbracket_{\text{ctx}}$ and $\llbracket \cdot \rrbracket_{\text{sig}}$ are straightforward, as all side-conditions directly match the definition of translatability.

For $\llbracket \cdot \rrbracket_{\text{exp}}$, $\llbracket \cdot \rrbracket_{\text{ty}}$ and $\llbracket \cdot \rrbracket_{\text{ter}}$, we will perform a case distinction on the expression e being translated:

If e is an application without top level **cast**, we are done by induction.

If e is a variable or constant, it $\llbracket e \rrbracket_{\text{exp}}$ translated to either $\llbracket e \rrbracket_{\text{ter}}$ or $\llbracket e \rrbracket_{\text{ty}}$, which both output e again.

If $e = e_1 =_{\alpha} e_2$, we know that $\Gamma \vdash_{\mathcal{T}} e : \mathbb{P}$, which means the third case of $\llbracket \cdot \rrbracket_{\text{exp}}$ applies. For $\llbracket \cdot \rrbracket_{\text{ter}}$, we are done by induction right away. For $\llbracket \cdot \rrbracket_{\text{ty}}$, e is not in the domain and we are done.

If $e = \lambda_{x:\alpha} e_1$, since by e being translatable we know that $\Gamma \vdash_{\mathcal{T}} e : \forall_{x:\alpha} \beta$ and $\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta \not\vdash \mathbb{U}_2$. From this we further know that $\beta \not\vdash \mathbb{U}_2$. This means either $\Gamma \vdash_{\mathcal{T}} \beta : \mathbb{U}_1$ or $\Gamma \vdash_{\mathcal{T}} \beta : \mathbb{P}$, meaning for $\llbracket \cdot \rrbracket_{\text{exp}}$ either case three or four apply. For $\llbracket \cdot \rrbracket_{\text{ter}}^{\Gamma; \mathcal{T}}$, we are again done by induction right away. For $\llbracket \cdot \rrbracket_{\text{ty}}$, e is not in the domain and we are done.

If $e = \forall_{x:\alpha} \beta$, since by e being translatable we know that $\Gamma \vdash_{\mathcal{T}} \forall_{x:\alpha} \beta \not\vdash \mathbb{U}_2$, either $\Gamma \vdash_{\mathcal{T}} e : \mathbb{U}_1$ or $\Gamma \vdash_{\mathcal{T}} e : \mathbb{P}$, meaning for $\llbracket \cdot \rrbracket_{\text{exp}}$ either case two or three apply. For $\llbracket \cdot \rrbracket_{\text{ter}}$, we know that $\Gamma \vdash_{\mathcal{T}} e : \mathbb{P}$, meaning $\Gamma \vdash_{\mathcal{T}} \beta : \mathbb{P}$. Then depending on the type of α , either the $\forall$ - or the $\Rightarrow$ -case applies. For $\llbracket \cdot \rrbracket_{\text{ty}}$, note that $\Gamma \vdash_{\mathcal{T}} \alpha \not\vdash \mathbb{P}$, due to e being translatable and $\Gamma \vdash_{\mathcal{T}} e : \mathbb{U}_1$ in order to be in the domain of $\llbracket \cdot \rrbracket_{\text{ty}}$. Therefore we are immediately done by induction.

If $e = \text{cast } p \ e_1$, then for $\llbracket \cdot \rrbracket_{\text{ty}}$ and $\llbracket \cdot \rrbracket_{\text{ter}}$ we are again done by induction. For $\llbracket \cdot \rrbracket_{\text{exp}}$, we perform a case distinction on e_1 as above. $\square$

Some natural examples of expressions that fall outside of the translatable fragment would be the polymorphic identity function, $\lambda_{x:\mathbb{U}_1} x$, or the function that returns the natural number n if it is equal to 0, $\lambda_{n:\mathbb{N}} \lambda_{p:n=\mathbb{N}_0} n$, as well as its type, $\forall_{n:\mathbb{N}} \forall_{p:n=\mathbb{N}_0} \mathbb{N}$. The first example can be somewhat side stepped by moving the polymorphic binding into the context, where type variables are allowed in DHOL, but this only helps in certain situations.

This fragment of DTT3 may seem restrictive, but it is still sufficient for many applications of DTT. In particular, higher rank polymorphism and higher universe levels are not needed to formalise many problems that do not relate to category theory or higher algebra.

Example 1. An axiomatisation of polymorphic length indexed vectors as a signature in DTT3 might look like this:

$$\mathcal{T} = (\mathbb{N}, \mathbb{U}_1), (0, \mathbb{N}), (\text{succ}, \forall_{n:\mathbb{N}} \mathbb{N}), (\text{Vec}, \forall_{\alpha:\mathbb{U}_1} \forall_{n:\mathbb{N}} \mathbb{U}_1), (\text{nil}, \forall_{\alpha:\mathbb{U}_1} \text{Vec } \alpha \ 0), \\ (\text{cons}, \forall_{\alpha:\mathbb{U}_1} \forall_{n:\mathbb{N}} \forall_{v:\text{Vec } \alpha \ n} \forall_{x:\alpha} \text{Vec } \alpha \ (\text{succ } n))$$

The translation of this signature is the following DHOL theory:

$$\llbracket \mathcal{T} \rrbracket_{\text{sig}} = \mathbb{N} : \text{type}, 0 : \mathbb{N}, \text{succ} : \Pi_{n:\mathbb{N}} \mathbb{N}, \text{Vec} : \Pi_{\alpha} \Pi_{n:\mathbb{N}} : \text{type}, \text{nil} : \Pi_{\alpha} \text{Vec } \alpha \ 0, \\ \text{cons} : \Pi_{\alpha} \Pi_{n:\mathbb{N}} \Pi_{v:\text{Vec } \alpha \ n} \Pi_{x:\alpha} \text{Vec } \alpha \ (\text{succ } n)$$

We can state the following formula in DTT3:

$$\alpha : \mathbf{U}_1 \vdash_{\mathcal{T}} \forall_{x:\alpha} \forall_{y:\alpha} \forall_{n:\mathbb{N}} \forall_{v:\mathbf{Vec} \alpha n} \forall_{p:\mathbf{cons} \alpha n v x =_{\mathbf{Vec} \alpha (\mathbf{suc} n)} \mathbf{cons} \alpha n v y} x =_{\alpha} y : \mathbb{P}$$

We can then translate the context, the signature, the proposition and its type, using $\llbracket \cdot \rrbracket_{\text{exp}}$ for the last two, as follows:

$$\begin{aligned} \alpha : \text{type} &\vdash_{\llbracket \mathcal{T} \rrbracket_{\text{sig}}} \\ \forall_{x:\alpha} \forall_{y:\alpha} \forall_{n:\mathbb{N}} \forall_{v:\mathbf{Vec} \alpha n} \mathbf{cons} \alpha n v x &=_{\mathbf{Vec} \alpha (\mathbf{suc} n)} \mathbf{cons} \alpha n v y \Rightarrow x =_{\alpha} y : o \end{aligned}$$

Notably, the translation maps the $\mathbb{P}$ -valued dependent function types of DTT3 to the $\forall$ quantifier in DHOL, which is defined in terms of equality and implication, instead of the DHOL dependent function type. Furthermore, since the last dependent function quantifies over a proposition, DHOL's dependent implication is used.

Example 2. This example demonstrates how casts are removed and how this interacts with provability. We reuse the length indexed vectors from above and assume a further axiomatisation of $+$: $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, which we write infix as usual, as well as of $\mathbf{append} : \forall_{\alpha:\mathbf{U}_1} \forall_{m:\mathbb{N}} \forall_{n:\mathbb{N}} \forall_{v:\mathbf{Vec} \alpha m} \forall_{w:\mathbf{Vec} \alpha n} \mathbf{Vec} \alpha (m+n)$ in $\mathcal{T}$. In DTT3 the following is ill-typed:

$$\begin{aligned} \alpha : \mathbf{U}_1 \vdash_{\mathcal{T}} \forall_{m:\mathbb{N}} \forall_{n:\mathbb{N}} \forall_{v:\mathbf{Vec} \alpha m} \forall_{w:\mathbf{Vec} \alpha n} \\ \mathbf{append} \alpha m n v w =_{\mathbf{Vec} \alpha (m+n)} \mathbf{append} \alpha n m w v : \mathbb{P} \end{aligned}$$

This is because $m+n$ and $n+m$ are propositionally equal but not definitionally equal. Therefore the types $\mathbf{Vec} \alpha (m+n)$ and $\mathbf{Vec} \alpha (n+m)$ are also not definitionally equal, which is required to even express propositional equality of the above vectors. If, however, we replace the right hand side of the equation by $\alpha : \mathbf{U}_1 \vdash_{\mathcal{T}} \mathbf{cast} p (\mathbf{append} \alpha n m w v)$, where p is an easy-to-construct proof term of type $\mathbf{Vec} \alpha (n+m) =_{\mathbf{U}_1} \mathbf{Vec} \alpha (m+n)$, this type-checks.

By contrast, the translated version of the above statement type-checks without issue, since DHOL is extensional. Specifically, while type checking, the rule TpEqSym would be used, where one of the typing conditions would be that $\Gamma \vdash_{\llbracket \mathcal{T} \rrbracket_{\text{sig}}} m+n =_{\mathbb{N}} n+m$, which is provable. For this reason, occurrences of $\mathbf{cast}$ are removed during the translation.

5 Soundness

A key theoretical property of our translation is that it reflects judgements: if some DHOL terms, types, theories or contexts satisfy a DHOL judgement, then there exist corresponding translatable DTT3 expressions, signatures and contexts of which the DHOL counterparts are translations and for which a corresponding DTT3 judgement holds. This entails soundness, meaning that if a translated DTT3 expression is provable in DHOL, the original expression is provable in DTT3 (i.e. a proof term exists).

To prove reflection of judgements, we first show a certain kind of injectivity of our translation. This is needed to identify DTT3 constructs that according to the induction hypothesis in the proof of the theorem translate to the same corresponding DHOL construct but originate from different applications of the induction hypothesis.

Lemma 1 (Injectivity). *The translation functions $\llbracket \cdot \rrbracket_{\text{ty}}$, $\llbracket \cdot \rrbracket_{\text{ter}}$, $\llbracket \cdot \rrbracket_{\text{exp}}$, $\llbracket \cdot \rrbracket_{\text{sig}}$ and $\llbracket \cdot \rrbracket_{\text{ctx}}$ are all injective on the translatable fragment of DTT3, where additionally there are (possibly trivial) casts in front of every subexpression. For DTT3 we use definitional equality $\equiv$; for DHOL we use syntactic equality $=$.*

Proof. We prove injectivity by mutual structural induction on the inputs, separating the cases by function. Since the DTT3 expressions have casts in front of every subexpression, which are stripped by the translation functions, two such DTT3 expressions that translate to the same DHOL type or term are definitionally equal if the remaining non-cast components are. For this the proofs which are the first argument of the casts are always definitionally equal, since either are proofs of the same proposition, so PI applies.

$\llbracket \cdot \rrbracket_{\text{ty}}$: Let α, β be translatable DTT3 expressions of type $\forall \vec{x}:\vec{\alpha} \mathbf{U}_1$. We show that if $\llbracket \alpha \rrbracket_{\text{ty}} = \llbracket \beta \rrbracket_{\text{ty}}$ then $\alpha \equiv \beta$:

If $\alpha = x$ or $\alpha = c$, the translation function is the identity, therefore the conclusion follows immediately.

If $\alpha = \mathbb{P}$, then $\llbracket \alpha \rrbracket_{\text{ty}} = o = \llbracket \beta \rrbracket_{\text{ty}}$. This immediately implies that $\beta = \mathbb{P}$, since $\mathbb{P}$ by induction is the only pre-image of o under $\llbracket \cdot \rrbracket_{\text{ty}}$.

If $\alpha = \alpha_1 \alpha_2$, we have $\llbracket \alpha \rrbracket_{\text{ty}} = \llbracket \alpha \rrbracket_{\text{exp}} = \llbracket \beta \rrbracket_{\text{exp}}$, which again by induction has only one pre-image and we conclude $\alpha \equiv \beta$ using the induction hypothesis regarding $\llbracket \cdot \rrbracket_{\text{exp}}$.

If $\alpha = \forall_{x:\alpha_1} \alpha_2$, then $\llbracket \alpha \rrbracket_{\text{ty}} = \Pi_{x:\llbracket \alpha_1 \rrbracket_{\text{ty}}} \llbracket \alpha_2 \rrbracket_{\text{ty}} = \llbracket \beta \rrbracket_{\text{ty}}$. Again by induction this only has one pre-image, therefore $\beta = \forall_{x:\alpha_1} \alpha_2$.

The case for $\alpha = \lambda_{x:\alpha_1} \alpha_2$ is completely analogous.

Expressions of the form $\alpha_1 =_{\alpha_2} \alpha_3$ are by definition of type $\mathbb{P}$ and can therefore not occur here.

$\llbracket \cdot \rrbracket_{\text{ter}}$: Let e, d be translatable DTT3 expressions of type $\forall \vec{x}:\vec{\beta} \alpha$, where $\alpha : \mathbf{U}_1$. We show that if $\llbracket e \rrbracket_{\text{ter}} = \llbracket d \rrbracket_{\text{ter}}$ then $e \equiv d$:

Cases $e = x, e = c, e = e_1 e_2$ are completely analogous to $\llbracket \cdot \rrbracket_{\text{ty}}$.

If $e = \exists_{x:\alpha} e_1$, then $\llbracket e \rrbracket_{\text{ter}} = \exists_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket e_1 \rrbracket_{\text{ter}}$. We use the induction hypothesis of the mutual induction regarding $\llbracket \cdot \rrbracket_{\text{ty}}$, which gets us that $\llbracket d \rrbracket_{\text{ter}} = \exists_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket e_1 \rrbracket_{\text{ter}}$. We conclude $e \equiv d$ from the definition of $\llbracket \cdot \rrbracket_{\text{ter}}$.

If $e = \forall_{x:\alpha} e_1$, we need to perform a case distinction on whether $\alpha : \mathbb{P}$ or not. If $\alpha : \mathbb{P}$ we have $\llbracket e \rrbracket_{\text{ter}} = \llbracket \alpha \rrbracket_{\text{ter}} \Rightarrow \llbracket e_1 \rrbracket_{\text{ter}}$. We use the induction hypothesis, which gets us that $\llbracket d \rrbracket_{\text{ter}} = \llbracket \alpha \rrbracket_{\text{ter}} \Rightarrow \llbracket e_1 \rrbracket_{\text{ter}}$. We conclude $e \equiv d$ from the definition of $\llbracket \cdot \rrbracket_{\text{ter}}$. If not, we have $\llbracket e \rrbracket_{\text{ter}} = \forall_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket e_1 \rrbracket_{\text{ter}}$ and the rest is analogous to the $\exists$ case.

For $e = (e_1 =_{\alpha} e_2)$ and $e = \lambda_{x:\alpha} e_1$ we employ the induction hypothesis of the mutual induction similarly.

Finally, $e = e_1 \wedge e_2, e = e_1 \vee e_2, e = \neg e_1$ all follow analogously to $e = (e_1 =_\alpha e_2)$.

$\llbracket \cdot \rrbracket_{\text{exp}}$: In the case of e being of the form $\text{cast } p \ e_1$, or it is not an application, this immediately follows from the injectivity of $\llbracket \cdot \rrbracket_{\text{ty}}$ and $\llbracket \cdot \rrbracket_{\text{ter}}$ as well as IP for the $\checkmark$ case. Otherwise, if e is an application without top-level cast , we invoke the induction hypothesis as before.

$\llbracket \cdot \rrbracket_{\text{sig}}$: Let $\mathcal{T}_1, \mathcal{T}_2$ be well formed DTT3 signatures. We show that if $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \llbracket \mathcal{T}_2 \rrbracket_{\text{sig}}$ then $\mathcal{T}_1 \equiv \mathcal{T}_2$:

If $\mathcal{T}_1 = \circ$, then $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \circ = \llbracket \mathcal{T}_2 \rrbracket_{\text{sig}}$ and trivially $\mathcal{T}_1 = \circ$.

If $\mathcal{T}_1 = \mathcal{T}'_1, (c, \forall_{x:U_1} e)$ we need to do a case distinction on e . First if $e : \mathbb{P}$, then $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \llbracket \mathcal{T}'_1 \rrbracket_{\text{sig}}, c \triangleright \Pi_{\vec{x}^n} \llbracket e \rrbracket_{\text{ter}}$. Using the induction hypothesis regarding $\llbracket \cdot \rrbracket_{\text{ter}}$, we get that this also equals $\llbracket \mathcal{T}_2 \rrbracket_{\text{sig}}$. From the definition of $\llbracket \cdot \rrbracket_{\text{sig}}$, since c does not have the val subscript, by induction there is only one possible pre-image, so we conclude that $\mathcal{T}_1 \equiv \mathcal{T}_2$.

Next, if $e : U_1$, then $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \llbracket \mathcal{T}'_1 \rrbracket_{\text{sig}}, c : \Pi_{\vec{x}^n} \llbracket e \rrbracket_{\text{ty}}$. Analogously to the previous case we use the induction hypothesis, regarding $\llbracket \cdot \rrbracket_{\text{ty}}$ this time, and then by induction there is only one pre-image, meaning $\mathcal{T}_1 \equiv \mathcal{T}_2$.

Finally, if e had neither type U_1 nor type $\mathbb{P}$, in our fragment it must have the form $\forall_{\vec{x}:\vec{\alpha}^m U_1}$, in which case $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \llbracket \mathcal{T}'_1 \rrbracket_{\text{sig}}, c : \Pi_{\vec{x}^n} \Pi_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \rightarrow_m \text{type}$. Using the induction hypothesis regarding $\llbracket \cdot \rrbracket_{\text{ty}}$ again and the definition of $\llbracket \cdot \rrbracket_{\text{sig}}$, we again conclude $\mathcal{T}_1 \equiv \mathcal{T}_2$.

If $\mathcal{T}_1 = \mathcal{T}'_1, (c, \alpha, e)$ then $\llbracket \mathcal{T}_1 \rrbracket_{\text{sig}} = \llbracket \mathcal{T}'_1 \rrbracket_{\text{sig}}, c_{\text{val}} \triangleright c =_{\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket e \rrbracket_{\text{ter}}$. Using the induction hypotheses regarding $\llbracket \cdot \rrbracket_{\text{ty}}$ and $\llbracket \cdot \rrbracket_{\text{ter}}$, we get that this also equals $\llbracket \mathcal{T}_2 \rrbracket_{\text{sig}}$. From the definition of $\llbracket \cdot \rrbracket_{\text{sig}}$, since c does have the val subscript, by induction there is only one possible pre-image, so we conclude that $\mathcal{T}_1 \equiv \mathcal{T}_2$.

$\llbracket \cdot \rrbracket_{\text{ctx}}$: Let Γ, Δ be well formed DTT3 contexts. We show that if $\llbracket \Gamma \rrbracket_{\text{ctx}} = \llbracket \Delta \rrbracket_{\text{ctx}}$ then $\Gamma \equiv \Delta$:

If $\Gamma = \circ$ we are again done analogously to before.

If $\Gamma = \Gamma', x : U_1$, then $\llbracket \Gamma \rrbracket_{\text{ctx}} = \llbracket \Gamma' \rrbracket_{\text{ctx}}, x : \text{type} = \llbracket \Delta \rrbracket_{\text{ctx}}$. There is exactly one pre-image and we conclude $\Gamma \equiv \Delta$.

If $\Gamma = \Gamma', x : \mathbb{P}$, then $\llbracket \Gamma \rrbracket_{\text{ctx}} = \llbracket \Gamma' \rrbracket_{\text{ctx}}, x : o = \llbracket \Delta \rrbracket_{\text{ctx}}$. Again from the definition of $\llbracket \cdot \rrbracket_{\text{ctx}}$ we conclude $\Gamma \equiv \Delta$.

If $\Gamma = \Gamma', x : e$, where e is neither $\mathbb{P}$ nor U_1 , we need to do a case distinction on the type of e . It must either be $\mathbb{P}$ or U_1 .

If $e : \mathbb{P}$, then $\llbracket \Gamma \rrbracket_{\text{ctx}} = \llbracket \Gamma' \rrbracket_{\text{ctx}}, x \triangleright \llbracket e \rrbracket_{\text{ter}}$. Using the mutual induction hypothesis regarding $\llbracket \cdot \rrbracket_{\text{ter}}$ we get that it is also equal to $\llbracket \Delta \rrbracket_{\text{ctx}}$. With the definition of $\llbracket \cdot \rrbracket_{\text{ctx}}$ we again conclude $\Gamma \equiv \Delta$.

If finally $e : U_1$, then $\llbracket \Gamma \rrbracket_{\text{ctx}} = \llbracket \Gamma' \rrbracket_{\text{ctx}}, x : \llbracket e \rrbracket_{\text{ty}}$. Analogous to the previous case, using the induction hypothesis regarding $\llbracket \cdot \rrbracket_{\text{ty}}$ and the definition of $\llbracket \cdot \rrbracket_{\text{ctx}}$ we conclude $\Gamma \equiv \Delta$. $\square$

We also need to prove that $\llbracket \cdot \rrbracket_{\text{exp}}$ is compatible with substitution, meaning that substituting in the DTT3 expression and then translating leads to the

same result as translating the expression that is substituted into as well as the expressions used in the substitution individually first, and performing the DHOL substitution after.

Lemma 2 (Substitution). *Let t be a translatable DTT3 expression, and let $\vec{e}$ be a list of translatable DTT3 expressions. Then $\llbracket t[\vec{e}] \rrbracket_{\text{exp}} = \llbracket t \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$.*

Proof. The proof is by induction on t :

$t = x_i$: Independent of the type of x_i , $\llbracket x_i \rrbracket_{\text{exp}} = x_i$. Therefore $\llbracket x_i[\vec{e}] \rrbracket_{\text{exp}} = \llbracket e_i \rrbracket_{\text{exp}} = x_i[\llbracket e_i \rrbracket_{\text{exp}}] = \llbracket x_i \rrbracket_{\text{exp}}[\llbracket e_i \rrbracket_{\text{exp}}] = \llbracket x_i \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$.

$t = \mathbb{P}$: Then $\llbracket t \rrbracket_{\text{exp}} = o$ and $t[\vec{e}] = t$, meaning $\llbracket t[\vec{e}] \rrbracket_{\text{exp}} = \llbracket t \rrbracket_{\text{exp}} = o = o[\llbracket \vec{e} \rrbracket_{\text{exp}}] = \llbracket t \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$.

$t = c$: Analogous to previous case.

$t = u_1 \ u_2$: Then $\llbracket t \rrbracket_{\text{exp}} = \llbracket u_1 \rrbracket_{\text{exp}} \ \llbracket u_2 \rrbracket_{\text{exp}}$ and also $t[\vec{e}] = u_1[\vec{e}] \ u_2[\vec{e}]$. So we get $\llbracket t[\vec{e}] \rrbracket_{\text{exp}} = \llbracket u_1[\vec{e}] \rrbracket_{\text{exp}} \ \llbracket u_2[\vec{e}] \rrbracket_{\text{exp}} \stackrel{I.H.}{=} \llbracket u_1 \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}] \ \llbracket u_2 \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}] = \llbracket t \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$.

$t = \lambda_{x:\alpha} u$: This means $t \not\vdash \mathbb{U}_1$, so $\llbracket t \rrbracket_{\text{exp}} = \llbracket t \rrbracket_{\text{ter}}$. Further $t[\vec{e}] = \lambda_{x:\alpha[\vec{e}]} u[\vec{e}]$. So we get $\llbracket t[\vec{e}] \rrbracket_{\text{ter}} = \lambda_{x:\llbracket \alpha[\vec{e}] \rrbracket_{\text{ty}}} \llbracket u[\vec{e}] \rrbracket_{\text{exp}} \stackrel{I.H.}{=} \lambda_{x:\llbracket \alpha \rrbracket_{\text{ty}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]} \llbracket u \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}] = \llbracket t \rrbracket_{\text{ter}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$. Since α must be of type $\mathbb{U}_1$, it is equivalent to use $\llbracket \cdot \rrbracket_{\text{ty}}$ instead of $\llbracket \cdot \rrbracket_{\text{exp}}$.

$t = \forall_{x:\alpha_1} u$: There are two cases to consider: $t : \mathbb{P}$ and $t : \mathbb{U}_1$. If $t : \mathbb{P}$, then $\llbracket t \rrbracket_{\text{exp}} = \llbracket t \rrbracket_{\text{ter}} = \forall_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket u \rrbracket_{\text{ter}}$ with $\alpha : \mathbb{U}_1$ and $u : \mathbb{P}$. Also $t[\vec{e}] = \forall_{x:\alpha[\vec{e}]} u[\vec{e}]$. So we get $\llbracket t[\vec{e}] \rrbracket_{\text{exp}} = \llbracket \forall_{x:\alpha[\vec{e}]} u[\vec{e}] \rrbracket_{\text{exp}} = \forall_{x:\llbracket \alpha[\vec{e}] \rrbracket_{\text{ty}}} \llbracket u[\vec{e}] \rrbracket_{\text{ter}} \stackrel{I.H.}{=} \forall_{x:\llbracket \alpha \rrbracket_{\text{ty}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]} \llbracket u \rrbracket_{\text{ter}}[\llbracket \vec{e} \rrbracket_{\text{exp}}] = \llbracket t \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$. If $t : \mathbb{U}_1$, then $\llbracket t \rrbracket_{\text{exp}} = \llbracket t \rrbracket_{\text{ty}} = \Pi_{x:\llbracket \alpha \rrbracket_{\text{ty}}} \llbracket u \rrbracket_{\text{ty}}$ with $\alpha : \mathbb{U}_1$ and $u : \mathbb{U}_1$. Also $t[\vec{e}] = \Pi_{x:\alpha[\vec{e}]} u[\vec{e}]$. So we get $\llbracket t[\vec{e}] \rrbracket_{\text{exp}} = \llbracket \Pi_{x:\alpha[\vec{e}]} u[\vec{e}] \rrbracket_{\text{exp}} = \Pi_{x:\llbracket \alpha[\vec{e}] \rrbracket_{\text{ty}}} \llbracket u[\vec{e}] \rrbracket_{\text{ty}} \stackrel{I.H.}{=} \Pi_{x:\llbracket \alpha \rrbracket_{\text{ty}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]} \llbracket u \rrbracket_{\text{ty}}[\llbracket \vec{e} \rrbracket_{\text{exp}}] = \llbracket t \rrbracket_{\text{exp}}[\llbracket \vec{e} \rrbracket_{\text{exp}}]$ $\square$

Next we state the reflection of judgements theorem. In general, we try to match the judgements of DHOL with those of DTT3. The cases where they notably differ are the ones for provability of a formula F and for the well-formedness of a substitution δ . For the former, we require the existence of a term of the corresponding proposition $\llbracket F \rrbracket_{\text{ter}}$ in DTT3. For the latter, since DTT3 does not have explicit substitutions, we spell out the condition each of the components of δ needs to fulfil individually. This works since δ can simply be viewed as the list $\vec{\delta}^n$ of terms, types and $\surd_F$'s for some DHOL context Δ with corresponding types, type variables and local assumptions $\vec{\Delta}^n$ for some n .

Table 2. Correspondence between DHOL and DTT3 judgements

DHOL	DTT3
$\Gamma \vdash_T^{\text{DHOL}} F$	There exists p such that $\Gamma' \vdash_{\mathcal{T}}^{\text{DTT3}} p : e$ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket e \rrbracket_{\text{ter}}^{\Gamma'; \mathcal{T}} = F, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\Gamma \vdash_T^{\text{DHOL}} t : A$	$\Gamma' \vdash_{\mathcal{T}}^{\text{DTT3}} e : \alpha$ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket e \rrbracket_{\text{ter}}^{\Gamma'; \mathcal{T}} = t, \llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma'; \mathcal{T}} = A, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\vdash^{\text{DHOL}} T \text{ Thy}$	$\vdash^{\text{DTT3}} \mathcal{T} \text{ sig}$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\vdash_T^{\text{DHOL}} \Gamma \text{ Ctx}$	$\vdash_{\mathcal{T}}^{\text{DTT3}} \Gamma' \text{ ok}$ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\Gamma \vdash_T^{\text{DHOL}} \Delta \leftarrow \delta$	$\Gamma' \vdash_{\mathcal{T}}^{\text{DTT3}} \delta'_i : \alpha[\delta^{\rightarrow i-1}]$ when $\Delta'_i = x : \alpha[\delta^{\rightarrow i-1}]$ for $1 \leq i \leq \delta $ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}}^{\Gamma'; \mathcal{T}} = \delta_i, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\Gamma \vdash_T^{\text{DHOL}} A : \text{type}$	$\Gamma' \vdash_{\mathcal{T}}^{\text{DTT3}} \alpha : \mathbf{U}_1$ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma'; \mathcal{T}} = A, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$
$\Gamma \vdash_T^{\text{DHOL}} A \equiv B$	There exists p such that $\Gamma' \vdash_{\mathcal{T}}^{\text{DTT3}} p : \alpha =_{\mathbf{U}_1} \beta$ and $\llbracket \Gamma' \rrbracket_{\text{ctx}}^{\mathcal{T}} = \Gamma, \llbracket \alpha \rrbracket_{\text{ty}}^{\Gamma'; \mathcal{T}} = A, \llbracket \beta \rrbracket_{\text{ty}}^{\Gamma'; \mathcal{T}} = B, \llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$

Theorem 2 (Reflection of Judgements). *Let Γ, Δ be a DHOL context, A, B be DHOL types, t be a DHOL term, F be a DHOL formula, δ be a DHOL substitution and T be a DHOL theory. In Table 2, if the left hand side holds, then there exist translatable DTT3 contexts Γ', Δ' , translatable DTT3 expressions $e, \alpha, \beta, \delta'_i$ and a translatable DTT3 signature $\mathcal{T}$ such that the right hand side also holds.*

We actually prove a slightly stronger version, where all DTT3 expressions have casts in front of every subexpression. This is necessary to fulfil the requirements of Lemma 1. The stronger condition can easily be fulfilled by adding casts that use a proof of reflexivity in front of subexpressions that do not have casts. We omit trivial casts for readability.

Since Lemma 1 only provides definitional equality of DTT3 constructs, we need to further invoke the compatibility of DTT3 judgements with definitional equality, i.e. subject reduction, meaning if some DTT3 judgement holds and there exist definitionally equal constructs, the same judgement also holds for these. This holds in the generalisation of DTT3 given by Carneiro [9], so it should hold for DTT3 as well.

For example regarding the typing judgement we have

$$\frac{\begin{array}{ccccc} \vdash \mathcal{T} \equiv \mathcal{T}' & \vdash_{\mathcal{T}} \Gamma \equiv \Gamma' & \Gamma \vdash_{\mathcal{T}} e \equiv e' \\ \Gamma \vdash_{\mathcal{T}} \alpha \equiv \alpha' & \vdash \mathcal{T} \text{ sig} & \vdash_{\mathcal{T}} \Gamma \text{ ok} & \Gamma \vdash_{\mathcal{T}} \alpha \text{ type} & \Gamma \vdash_{\mathcal{T}} e : \alpha \end{array}}{\Gamma' \vdash_{\mathcal{T}'} e' : \alpha'}$$

Similar statements for all other judgements hold.

Proof. We will use induction on the DHOL proof of our assumption.

Case ThyEmpty: Follows trivially from the first case of the sig judgement.

Case ThyTp: The assumptions of the rule are

$$(1) \vdash^{\text{DHOL}} T \text{ Thy}, (2) a \notin T, (3) \vdash_T^{\text{DHOL}} \Delta \text{ Ctx}, (4) \Delta = \vec{\alpha} : \text{type}, \vec{x} : \vec{A}$$

The conclusion is $(5) \vdash^{\text{DHOL}} T, a : \Pi_{\Delta} : \text{type} \text{ Thy}$. The induction hypothesis allows us to use the assumptions (1) and (3) to derive that there exists a DTT3 signature $\mathcal{T}$ and a DTT3 context Δ' , which have casts in front of every subexpression, such that $(1') \vdash^{\text{DTT3}} \mathcal{T} \text{ sig}$ where $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$ and $(3') \vdash_{\mathcal{T}}^{\text{DTT3}} \Delta' \text{ ok}$ where $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta$. Note that we implicitly use the injectivity of the translation function stated in Lemma 2 to equate the $\mathcal{T}$ in (1') and (3'). To prove this case of the theorem, we now need to show given (1'), (2), (3'), (4) that there exist a DTT3 signature $\mathcal{T}'$ such that if (5) holds, it holds that $(5') \vdash^{\text{DTT3}} \mathcal{T}' \text{ sig}$ with $\llbracket \mathcal{T}' \rrbracket_{\text{sig}} = T, a : \Pi_{\Delta} : \text{type}$. We therefore assume (5) and show (5'). We choose $\mathcal{T}' = \mathcal{T}, (a, \forall_{\Delta'} U_1)$, which translates to the theory $\llbracket \mathcal{T}' \rrbracket_{\text{sig}}, a : \Pi_{\llbracket \Delta' \rrbracket_{\text{ctx}}} : \text{type}$, which is equal to $T, a : \Pi_{\Delta} : \text{type}$ due to the equalities associated with (1') and (3'). It remains to prove $\vdash^{\text{DTT3}} \mathcal{T}', (a, \forall_{\Delta'} U_1) \text{ sig}$. For this it suffices to show that $\vdash_{\mathcal{T}'}^{\text{DTT3}} \forall_{\Delta'} U_1 \text{ type}$. From the definition of the translation function, (4) and the equalities associated with (3') we know that $\Delta' = \vec{\alpha} : U_1, \vec{x} : \vec{\beta}$, where $\llbracket \beta_i \rrbracket_{\text{ty}} = A_i$. To show our goal we use $\forall \text{Form}$. It therefore suffices that $\vec{\alpha} : U_1, \vec{x}^{i-1} : \vec{\beta}^{i-1} \vdash_{\mathcal{T}'}^{\text{DTT3}} \beta_i \text{ type}$ for all β_i . This then follows from (3'), which concludes the case.

In the following cases we will only state the results of applying the induction hypothesis to the DHOL pre-conditions (here, (1'), (3')), possible leftover assumptions (here, (2), (4)), the collection of equalities associated with these and state directly what we must prove (here, (5')), without necessarily mentioning the DHOL conclusion directly.

Case ThyCon: From the induction hypothesis, we get that $(1) \vdash \mathcal{T} \text{ sig}$ and $(2) \Delta' \vdash_{\mathcal{T}} \beta : U_1$ with $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta$ and $\llbracket \beta \rrbracket_{\text{ty}} = A$. We further get the assumptions $c \notin T$ and $(3) \Delta = \vec{\alpha} : \text{type}$. We need to show that $\vdash \mathcal{T}' \text{ sig}$, where $\llbracket \mathcal{T}' \rrbracket_{\text{sig}} = T, c : \Pi_{\Delta} A$. We choose $\mathcal{T}' = \mathcal{T}, (c, \forall_{\Delta'} \beta)$. $\llbracket \mathcal{T}' \rrbracket_{\text{sig}}$ computes as desired. It suffices to show $\vdash_{\mathcal{T}'} \forall_{\Delta'} \beta \text{ type}$. We use $\forall \text{Form}$, so we need to show $\Delta' \vdash_{\mathcal{T}'} \beta \text{ type}$, which follows from (2), and $\vec{x}^{i-1} : \vec{\alpha}^{i-1} \vdash_{\mathcal{T}'} \alpha'_i \text{ type}$ for all α'_i in Δ' . We know $\Delta' = \vec{\alpha}' : U_1$, due to (3) and the definition of the translation function. From this the desired result follows immediately.

Case ThyAsn: From the induction hypothesis, we get that $(1) \vdash \mathcal{T} \text{ sig}$ and $(2) \Delta' \vdash_{\mathcal{T}} e : \beta$ with $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket e \rrbracket_{\text{ter}} = F$ and $\llbracket \beta \rrbracket_{\text{ty}} = o$. We further get the assumptions $c \notin T$ and $(3) \Delta = \vec{\alpha} : \text{type}$. We need to show that $\vdash \mathcal{T}' \text{ sig}$, where $\llbracket \mathcal{T}' \rrbracket_{\text{sig}} = T, c \triangleright \Pi_{\Delta} F$. We choose $\mathcal{T}' = \mathcal{T}, (c, \forall_{\Delta'} e)$. $\llbracket \mathcal{T}' \rrbracket_{\text{sig}}$ computes as desired. The rest of the proof is analogous to the previous case, apart from showing $\vdash_{\mathcal{T}'} e \text{ type}$, which follows from (2) and from $\beta = \mathbb{P}$, due to $\llbracket \beta \rrbracket_{\text{ty}} = o$ and the definition of the translation function.

Case CtxEmpty: Follows trivially from the first case of the ok judgement.

Case CtxTp: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Delta'$ ok with $\llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\vdash_{\mathcal{T}} \Delta''$ ok with $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \Delta, \alpha : \text{type}$. We choose $\Delta'' = \Delta', \alpha : \mathbf{U}_1$. $\llbracket \Delta'' \rrbracket_{\text{ctx}}$ computes as desired. We then only need to show that $\vdash_{\mathcal{T}} \mathbf{U}_1$ type. This is then immediate.

Case CtxVar: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Delta'$ ok and $\Delta' \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$ with $\llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \alpha \rrbracket_{\text{ty}} = A$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\vdash_{\mathcal{T}} \Delta''$ ok with $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \Delta, x : A$. We choose $\Delta'' = \Delta', x : \alpha$. $\llbracket \Delta'' \rrbracket_{\text{ctx}}$ computes as desired. Thus we need to show $\vdash_{\mathcal{T}} \Delta'$ ok and $\Delta' \vdash_{\mathcal{T}} \alpha$ type, which follows immediately from our assumptions.

Case CtxAsn: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Delta'$ ok and $\Delta' \vdash_{\mathcal{T}} e : \alpha$ with $\llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket e \rrbracket_{\text{ter}} = F, \llbracket \alpha \rrbracket_{\text{ty}} = o$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\vdash_{\mathcal{T}} \Delta''$ ok with $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \Delta', x \triangleright F$. We choose $\Delta'' = \Delta', x : e$. $\llbracket \Delta'' \rrbracket_{\text{ctx}}$ computes as desired. Thus we need to show $\vdash_{\mathcal{T}} \Delta'$ ok and $\Delta' \vdash_{\mathcal{T}} e$ type. The former we already have as an assumption, the latter holds due to $\llbracket \alpha \rrbracket_{\text{ty}} = o$, which means that $\alpha = \mathbb{P}$.

Case SubEmpty: Trivial.

Case SubTp: From the induction hypothesis, we get (1) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \beta[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \beta[\overrightarrow{\delta'}^{i-1}]$ and (2) $\Gamma' \vdash_{\mathcal{T}} \gamma : \mathbf{U}_1$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i, \llbracket \gamma \rrbracket_{\text{ty}} = A$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\Gamma' \vdash_{\mathcal{T}} \delta''_i : \beta[\overrightarrow{\delta''}^{i-1}]$ when $\Delta''_i = x : \beta[\overrightarrow{\delta''}^{i-1}]$ where $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \Delta, \alpha : \text{type}$ and $\llbracket \delta''_i \rrbracket_{\text{exp}} = \delta_i$ for all $1 \leq i \leq |\delta|$ and $\llbracket \delta''_{|\delta|+1} \rrbracket_{\text{exp}} = A$. We choose $\delta''_i = \delta'_i$ for $1 \leq i \leq |\delta|$ and $\delta''_{|\delta|+1} = \gamma$ and $\Delta'' = \Delta', \alpha : \mathbf{U}_1$. $\llbracket \delta''_i \rrbracket_{\text{exp}}$ and $\llbracket \Delta'' \rrbracket_{\text{ctx}}$ compute as desired. For all $1 \leq i \leq |\delta|$, our goal is equal to (1). For $i = |\delta| + 1$, we need to show $\Gamma' \vdash_{\mathcal{T}} \gamma : \mathbf{U}_1$, which is exactly (2).

Case SubVar: From the induction hypothesis, we get (1) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \alpha[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \alpha[\overrightarrow{\delta'}^{i-1}]$ and (2) $\Delta' \vdash_{\mathcal{T}} \beta : \mathbf{U}_1$ and (3) $\Gamma' \vdash_{\mathcal{T}} e : \gamma$ with $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i, \llbracket \beta \rrbracket_{\text{ty}} = A, \llbracket \gamma \rrbracket_{\text{ty}} = A[\delta], \llbracket e \rrbracket_{\text{ter}} = t$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. The goal and choice is analogous to the previous case, with the exception of $\delta''_{|\delta|+1} = e$ and $\Delta'' = \Delta', x : \beta$. $\llbracket \delta''_i \rrbracket_{\text{exp}}$ and $\llbracket \Delta'' \rrbracket_{\text{ctx}}$ compute as desired. For all $1 \leq i \leq |\delta|$, our goal is equal to (1). For $i = |\delta| + 1$, we need to show $\Gamma' \vdash_{\mathcal{T}} e : \beta[\overrightarrow{\delta'}^{(|\delta|+1)-1}]$, which is equal to $\Gamma' \vdash_{\mathcal{T}} e : \beta[\delta']$. Using the induction hypotheses and Lemma 2 this then follows immediately.

Case SubAsn: From the induction hypothesis, we get (1) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \alpha[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \alpha[\overrightarrow{\delta'}^{i-1}]$ and (2) $\Delta' \vdash_{\mathcal{T}} e : \beta$ and there exists p such that (3) $\Gamma' \vdash_{\mathcal{T}} p : e'$ with $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i, \llbracket \beta \rrbracket_{\text{ty}} = o, \llbracket e \rrbracket_{\text{ter}} = F, \llbracket e' \rrbracket_{\text{ter}} = F[\delta]$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\Gamma' \vdash_{\mathcal{T}} \delta''_i : \beta[\overrightarrow{\delta''}^{i-1}]$ when $\Delta''_i = x : \beta[\overrightarrow{\delta''}^{i-1}]$ where $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \Delta, \alpha : \text{type}$ and $\llbracket \delta''_i \rrbracket_{\text{exp}} = \delta_i$ for all

$1 \leq i \leq |\delta|$ and $\llbracket \delta''_{|\delta|+1} \rrbracket_{\text{exp}} = p$. We choose $\delta''_i = \delta'_i$ for $1 \leq i \leq |\delta|$ and $\delta''_{|\delta|+1} = p$ and $\Delta'' = \Delta', x : e$. $\llbracket \delta''_i \rrbracket_{\text{exp}} = \delta_i$, for $1 \leq i \leq |\delta|$, and $\llbracket \delta''_{|\delta|+1} \rrbracket_{\text{exp}} = \check{\llbracket e' \rrbracket_{\text{ter}}} = \check{e}$. Further $\llbracket \Delta'' \rrbracket_{\text{ctx}} = \llbracket \Delta' \rrbracket_{\text{ctx}}, x \triangleright e = \Delta, x \triangleright e$. For all $1 \leq i \leq |\delta|$, our goal is equal to (1). For $i = |\delta| + 1$ we need to show $\Gamma' \vdash_{\mathcal{T}} p : e[\overrightarrow{\delta''}^{(|\delta|+1)-1}]$. This is equivalent to $\Gamma' \vdash_{\mathcal{T}} p : e[\delta']$. Using the induction hypotheses and Lemma 2 this then follows immediately.

Case TpSym: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Gamma'$ ok and (2) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \alpha[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \alpha[\overrightarrow{\delta'}^{i-1}]$ with $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (3) $a : \Pi_{\Delta} : \text{type}$ is in T . We need to show that $\Gamma' \vdash_{\mathcal{T}} \beta : \mathbf{U}_1$ where $\llbracket \beta \rrbracket_{\text{ty}} = a \delta$. We choose $\beta = a \delta'$. $\llbracket \beta \rrbracket_{\text{ty}}$ computes as desired. From the definition of the translation function and (3), we know that $(a, \forall_{\Delta} \mathbf{U}_1) \in \mathcal{T}$. This, in combination with (2), then yields the desired result using **Apply** and **ConstTy**.

Case TermSym: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Gamma'$ ok and (2) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \alpha[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \alpha[\overrightarrow{\delta'}^{i-1}]$ with $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (3) $c : \Pi_{\Delta} B$ is in T . We need to show that $\Gamma' \vdash_{\mathcal{T}} e : \beta$ where $\llbracket e \rrbracket_{\text{ter}} = c \delta$ and $\llbracket \beta \rrbracket_{\text{ty}} = B[\delta]$. From the definition of the translation function and (3), we know that $(c, \forall_{\Delta} \beta') \in \mathcal{T}$ where $\llbracket \beta' \rrbracket_{\text{ty}} = B$. We therefore choose $e = c \delta'$ and $\beta = \beta'[\delta']$. $\llbracket \beta \rrbracket_{\text{ty}}$ and $\llbracket e \rrbracket_{\text{ter}}$ compute as desired using Lemma 2. The desired result follows with (2) using **Apply** and **ConstTy**.

Case ValidSym: From the induction hypothesis, we get $\vdash_{\mathcal{T}} \Gamma'$ ok and (2) $\Gamma' \vdash_{\mathcal{T}} \delta'_i : \alpha[\overrightarrow{\delta'}^{i-1}]$ when $\Delta'_i = x : \alpha[\overrightarrow{\delta'}^{i-1}]$ with $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \delta'_i \rrbracket_{\text{exp}} = \delta_i$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (3) $c \triangleright \Pi_{\Delta} F$ is in T . We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e$ and $\llbracket e \rrbracket_{\text{ter}} = F[\delta]$. From the definition of the translation function and (3), we know that $(c, \forall_{\Delta} e') \in \mathcal{T}$ where $\llbracket e' \rrbracket_{\text{ter}} = F$. We therefore choose $e = e'[\delta']$ and $p = c \delta'$. $\llbracket e \rrbracket_{\text{ter}}$ computes as desired using Lemma 2. The desired result follows with (2) using **Apply** and **ConstTy**.

Case TpVar: From the induction hypothesis, we have $\vdash_{\mathcal{T}} \Gamma'$ ok where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (2) $\alpha : \text{type} \in \Gamma$. From this and the definition of the translation function we therefore know that (3) $\alpha : \mathbf{U}_1 \in \Gamma'$. We need to show that $\Gamma' \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$. This follows from the **Assume** rule and (3).

Case TermVar: Analogous to **TpVar**.

Case ValidVar: From the induction hypothesis, we have $\vdash_{\mathcal{T}} \Gamma'$ ok where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (2) $x \triangleright F \in \Gamma$. From this and the definition of the translation function we therefore know that (3) $x : e \in \Gamma'$ where $\llbracket e \rrbracket_{\text{ter}} = F$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e'$ and $\llbracket e' \rrbracket_{\text{ter}} = F$. We choose $e' = e$ and $p = x$. The result then follows from the **Assume** rule and (3).

Case TpBool: Follows trivially from the **Univ** rule.

Case TermEq: From the induction hypothesis, we get that $\Gamma' \vdash_{\mathcal{T}} t' : \alpha$ and $\Gamma' \vdash_{\mathcal{T}} u' : \alpha$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket t' \rrbracket_{\text{ter}} = t$, $\llbracket u' \rrbracket_{\text{ter}} = u$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\Gamma' \vdash_{\mathcal{T}} e : \beta$ where $\llbracket e \rrbracket_{\text{ter}} = (t =_A u)$ and $\llbracket \beta \rrbracket_{\text{ty}} = o$. We choose $e = (t' =_{\alpha} u')$ and $\beta = \mathbb{P}$. $\llbracket e \rrbracket_{\text{ter}}$ and $\llbracket \beta \rrbracket_{\text{ty}}$ compute as desired. The result then immediately follows from the induction hypotheses and **EqForm**.

Case TermImpl: From the induction hypothesis, we have (1) $\Gamma' \vdash_{\mathcal{T}} e : \alpha$ and (2) $\Gamma'' \vdash_{\mathcal{T}} u : \alpha$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x \triangleright F$, $\llbracket e \rrbracket_{\text{ter}} = F$, $\llbracket u \rrbracket_{\text{ter}} = G$, $\llbracket \alpha \rrbracket_{\text{ty}} = o$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\Gamma'' = \Gamma', x : e$ and $\alpha = \mathbb{P}$. We need to show that $\Gamma' \vdash_{\mathcal{T}} t : \beta$ where $\llbracket t \rrbracket_{\text{ter}} = F \Rightarrow G$ and $\llbracket \beta \rrbracket_{\text{ty}} = o$. We choose $t = \forall_{x:e} u$ and $\beta = \alpha$. $\llbracket t \rrbracket_{\text{ter}}$ and $\llbracket \beta \rrbracket_{\text{ty}}$ compute as desired. We prove $\Gamma' \vdash_{\mathcal{T}} \forall_{x:e} u : \mathbb{P}$. We use **$\forall$ Form**, meaning it suffices to show $\Gamma' \vdash_{\mathcal{T}} e \text{ type}$ and $\Gamma', x : e \vdash_{\mathcal{T}} u : \mathbb{P}$ due to the definition of **imax**. Using (1) and (2) respectively with the fact that $\alpha = \mathbb{P}$ these follow immediately.

Case TpPi: From the induction hypothesis, we have $\Gamma' \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$ and $\Gamma'' \vdash_{\mathcal{T}} \beta : \mathbf{U}_1$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x : A$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$, $\llbracket \beta \rrbracket_{\text{ty}} = B$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\Gamma'' = \Gamma', x : \alpha$. We need to show that $\Gamma' \vdash_{\mathcal{T}} \gamma : \mathbf{U}_1$ where $\llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B$. We choose $\gamma = \forall_{x:\alpha} \beta$. $\llbracket \gamma \rrbracket_{\text{ty}}$ computes as desired. Using **$\forall$ Form**, it suffices to show that $\Gamma' \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$ and $\Gamma', x : \alpha \vdash_{\mathcal{T}} \beta : \mathbf{U}_1$, which is exactly the induction hypotheses.

Case TermLambda: From the induction hypothesis, we have $\Gamma' \vdash_{\mathcal{T}} \alpha : \mathbf{U}_1$ and (2) $\Gamma'' \vdash_{\mathcal{T}} t' : \beta$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x : A$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$, $\llbracket \beta \rrbracket_{\text{ty}} = B$, $\llbracket t' \rrbracket_{\text{ter}} = t$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\Gamma'' = \Gamma', x : \alpha$. We need to show that $\Gamma' \vdash_{\mathcal{T}} e : \gamma$ where $\llbracket e \rrbracket_{\text{ter}} = \lambda_{x:A} t$ and $\llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B$. We choose $e = \lambda_{x:\alpha} t'$ and $\gamma = \forall_{x:\alpha} \beta$. Using **λ Form** we need to show $\Gamma', x : \alpha \vdash_{\mathcal{T}} t' : \beta$, which is exactly (2).

Case TermApply: From the induction hypothesis, we have $\Gamma' \vdash_{\mathcal{T}} t' : \gamma$ and $\Gamma' \vdash_{\mathcal{T}} u' : \alpha$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$, $\llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B$, $\llbracket t' \rrbracket_{\text{ter}} = t$, $\llbracket u' \rrbracket_{\text{ter}} = u$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we also know that $\gamma = \forall_{x:\alpha} \beta$, where $\llbracket \beta \rrbracket_{\text{ty}} = B$. We need to show that $\Gamma' \vdash_{\mathcal{T}} e : \beta'$ where $\llbracket e \rrbracket_{\text{ter}} = t u$, $\llbracket \beta' \rrbracket_{\text{ty}} = B[u]$. We choose $e = t' u'$ and $\beta' = \beta[u]$. $\llbracket \beta' \rrbracket_{\text{ty}}$ and $\llbracket e \rrbracket_{\text{ter}}$ compute as desired using Lemma 2. Using **Apply**, it suffices to show $\Gamma' \vdash_{\mathcal{T}} t' : \forall_{x:\alpha} \beta$ and $\Gamma' \vdash_{\mathcal{T}} u' : \alpha$, which are exactly the induction hypotheses.

Case TpEqVar: Follows trivially from **EqIntro**.

Case TpEqSym: From the induction hypothesis, we have $\vdash_{\mathcal{T}} \Gamma' \text{ ok}$ and that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : \gamma_i =_{\Delta_i[\vec{\gamma}^{i-1}]} \gamma'_i$ since $\equiv_{\Delta}$ means the component-wise equality of the terms/types, where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta$, $\llbracket \gamma_i \rrbracket_{\text{exp}} = \delta_i$, $\llbracket \gamma'_i \rrbracket_{\text{exp}} = \delta'_i$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We also get the assumption that (3) $a : \Pi_{\Delta} : \text{type}$ is in T . From the definition of the translation function we then know that

$(a, \forall_{\Delta} \mathbf{U}_1) \in \mathcal{T}$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p' : \alpha =_{\mathbf{U}_1} \beta$ and $\llbracket \alpha \rrbracket_{\text{ty}} = a \ \delta$ and $\llbracket \beta \rrbracket_{\text{ty}} = a \ \delta'$. We choose $\alpha = a \ \gamma, \beta = a \ \gamma'$. $\llbracket \alpha \rrbracket_{\text{ty}}$ and $\llbracket \beta \rrbracket_{\text{ty}}$ compute as desired. Using the congruence rules about equality derivable in DTT3 and the induction hypotheses we can construct p' .

Case $TPEqBool$: Follows trivially from the introduction rule of equality.

Case $TPEqPi$: From the induction hypothesis, we have that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : \alpha =_{\mathbf{U}_1} \alpha'$ and that there exists p_2 such that $(2)\Gamma'' \vdash_{\mathcal{T}} p_2 : \beta =_{\mathbf{U}_1} \beta'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x : A, \llbracket \alpha \rrbracket_{\text{ty}} = A, \llbracket \alpha' \rrbracket_{\text{ty}} = A', \llbracket \beta \rrbracket_{\text{ty}} = B, \llbracket \beta' \rrbracket_{\text{ty}} = B'$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\Gamma'' = \Gamma', x : \alpha$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : \gamma =_{\mathbf{U}_1} \gamma'$ and $\llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B$ and $\llbracket \gamma' \rrbracket_{\text{ty}} = \Pi_{x:A'} B$. We choose $\gamma = \forall_{x:\alpha} \beta$ and $\gamma' = \forall_{x:\alpha'} \beta'$. After this everything type-checks. $\llbracket \gamma \rrbracket_{\text{ty}}$ and $\llbracket \gamma' \rrbracket_{\text{ty}}$ compute as desired. Using the common derivable rules for equality in DTT3 and the induction hypotheses we can construct p .

Case $TermConvert$: From the induction hypothesis, we have $\Gamma' \vdash_{\mathcal{T}} t' : \alpha$ and that there exists p such that $(2)\Gamma' \vdash_{\mathcal{T}} p : \alpha =_{\mathbf{U}_1} \alpha'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \Delta' \rrbracket_{\text{ctx}} = \Delta, \llbracket \alpha \rrbracket_{\text{ty}} = A, \llbracket \alpha' \rrbracket_{\text{ty}} = A', \llbracket t' \rrbracket_{\text{ter}} = t$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. We need to show that $\Gamma' \vdash_{\mathcal{T}} e : \beta$ where $\llbracket e \rrbracket_{\text{ter}} = t$ and $\llbracket \beta \rrbracket_{\text{ty}} = A'$. We choose $e = t''$ and $\beta = \alpha'$. t'' is t' where the top-level cast is replaced by one that uses p . $\llbracket e \rrbracket_{\text{ter}}$ and $\llbracket \beta \rrbracket_{\text{ty}}$ compute as desired. Due to the replacement of the cast our goal follows immediately.

Case $TermBeta$: From the induction hypothesis, we get $\Gamma' \vdash_{\mathcal{T}} e : \gamma$ and $\Gamma' \vdash_{\mathcal{T}} u' : \alpha$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B, \llbracket \alpha \rrbracket_{\text{ty}} = A, \llbracket e \rrbracket_{\text{ter}} = \lambda_{x:A} t, \llbracket u' \rrbracket_{\text{ter}} = u$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\gamma = \forall_{x:\alpha} \beta$ where $\llbracket \beta \rrbracket_{\text{ty}} = B$ and $e = \lambda_{x:\alpha} t'$ where $\llbracket t' \rrbracket_{\text{ter}} = t$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e'$ and $\llbracket e' \rrbracket_{\text{ter}} = (\lambda_{x:A} t) u =_{B[u]} t[u]$. We choose $e' = (\lambda_{x:\alpha} t') u' =_{\beta[u']} t'[u']$ and $p = \text{refl}_{t'[u']}$. $\llbracket e' \rrbracket_{\text{ter}}$ computes as desired using Lemma 2. This holds definitionally in DTT3 so it is proved by *refl*.

Case $TermEta$: From the induction hypothesis, we get $\Gamma' \vdash_{\mathcal{T}} t' : \gamma$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket \gamma \rrbracket_{\text{ty}} = \Pi_{x:A} B, \llbracket t' \rrbracket_{\text{ter}} = t$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\gamma = \forall_{x:\alpha} \beta$ where $\llbracket \alpha \rrbracket_{\text{ty}} = A$ and $\llbracket \beta \rrbracket_{\text{ty}} = B$. We additionally get the assumption that x does not occur in Γ . It therefore can also not occur in Γ' or we can perform α -renaming to achieve this. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e$ and $\llbracket e \rrbracket_{\text{ter}} = t =_{\Pi_{x:A} B} \lambda_{x:A} t x$. We choose $e = t' =_{\gamma} \lambda_{x:\alpha} t' x$ and $p = \text{refl}_{t'}$. $\llbracket e \rrbracket_{\text{ter}}$ computes as desired. Again this equation holds definitionally in DTT3 so it is proved by *refl*.

Case $CongApply$: From the induction hypothesis, we get that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : e$ and that there exists p such that $\Gamma \vdash_{\mathcal{T}} p_2 : e'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, \llbracket e \rrbracket_{\text{ter}} = t =_A t', \llbracket e' \rrbracket_{\text{ter}} = f =_{\Pi_{x:A} B} f'$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of

the translation function we further know that $e = (u =_{\alpha} u')$ and $e' = (g =_{\forall x:\alpha\beta} g')$ where $\llbracket u \rrbracket_{\text{ter}} = t$, $\llbracket u' \rrbracket_{\text{ter}} = t'$, $\llbracket g \rrbracket_{\text{ter}} = f$, $\llbracket g' \rrbracket_{\text{ter}} = f'$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$ and $\llbracket \beta \rrbracket_{\text{ty}} = B$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e''$ and $\llbracket e'' \rrbracket_{\text{ter}} = (f \ t =_{B[t]} f' \ t')$. We choose $e'' = (g \ u =_{\beta[u]} \text{cast } p'_1 \ (g' \ u'))$. The **cast** added around the right hand side of the equation is needed to make this statement type-check. The proof term $p'_1 : \beta[u] = \beta[u']$ can easily be derived using p_1 and the rules for equality. Then $\llbracket e'' \rrbracket_{\text{ter}}$ computes as desired using Lemma 2. Then using induction twice yields the desired p .

Case CongLambda: From the induction hypothesis, we get that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : \alpha =_{U_1} \alpha'$ and that there exists p_2 such that $\Gamma'' \vdash_{\mathcal{T}} p_2 : e$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x : A$, $\llbracket \alpha \rrbracket_{\text{ty}} = A$, $\llbracket \alpha' \rrbracket_{\text{ty}} = A'$, $\llbracket e \rrbracket_{\text{ter}} = t =_B t'$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $e = u =_{\beta} u'$ and $\Gamma'' = \Gamma', x : \alpha$ where $\llbracket u \rrbracket_{\text{ter}} = t$, $\llbracket u' \rrbracket_{\text{ter}} = t'$ and $\llbracket \beta \rrbracket_{\text{ty}} = B$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e'$ and $\llbracket e' \rrbracket_{\text{ter}} = \lambda_{x:A} t =_{\Pi_{x:A} B} \lambda_{x:A'} t'$. We choose $e' = \lambda_{x:\alpha} u =_{\forall x:\alpha\beta} \lambda_{x:\alpha'} u'$. $\llbracket e \rrbracket_{\text{ter}}$ computes as desired. To make this type-check, all occurrences of x in u' must have their casts amended by p_1 . Then using induction twice yields the desired p .

Case Refl: Follows trivially from the introduction rule of equality.

Case Sym: From the induction hypothesis, we get there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket e \rrbracket_{\text{ter}} = t =_A s$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $e = t' =_{\alpha} s'$ with $\llbracket t' \rrbracket_{\text{ter}} = t$, $\llbracket s' \rrbracket_{\text{ter}} = s$ and $\llbracket \alpha \rrbracket_{\text{ty}} = A$. We need to show that there exists p' such that $\Gamma' \vdash_{\mathcal{T}} p' : e'$ and $\llbracket e' \rrbracket_{\text{ter}} = s =_A t$. We choose $e' = t' =_{\alpha} s'$. $\llbracket e' \rrbracket_{\text{ter}}$ computes as desired. Using induction on p we can construct the desired p' .

Case CongValid: From the induction hypothesis, we get that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : e$ and that there exists p_2 such that $\Gamma' \vdash_{\mathcal{T}} p_2 : e'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket e \rrbracket_{\text{ter}} = F =_o F'$, $\llbracket e' \rrbracket_{\text{ter}} = F'$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $e = t =_{\mathbb{P}} e'$ where $\llbracket t \rrbracket_{\text{ter}} = F$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : t$. We can construct p by changing the cast around p_2 to one that casts along p_1 .

Case ImplIntro: From the induction hypothesis, we get that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma, x \triangleright F$, $\llbracket e \rrbracket_{\text{ter}} = G$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\Gamma' = \Gamma'', x : e'$ where $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma$ and $\llbracket e' \rrbracket_{\text{ter}} = F$. We need to show that there exists p' such that $\Gamma'' \vdash_{\mathcal{T}} p' : u$ and $\llbracket u \rrbracket_{\text{ter}} = F \Rightarrow G$. We choose $u = \forall_{x:e'} e$ and $p' = \lambda_{x:e'} p$. $\llbracket u \rrbracket_{\text{ter}}$ computes as desired. Using λForm with the induction hypothesis immediately yields the result.

Case MP: From the induction hypothesis, we get that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : e$ and that there exists p_2 such that $\Gamma' \vdash_{\mathcal{T}} p_2 : e'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket e \rrbracket_{\text{ter}} = F \Rightarrow G$, $\llbracket e' \rrbracket_{\text{ter}} = F$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the

translation function we further know that $e = \forall_{x:e'} e''$ where $\llbracket e'' \rrbracket_{\text{ter}} = G$. We need to show that there exists p such that $\Gamma' \vdash_{\mathcal{T}} p : e''$. We choose $p = p_1 \ p_2$. Using **Apply** with the induction hypotheses immediately yields the result.

Case BoolExt: From the induction hypothesis, we get $\Gamma' \vdash_{\mathcal{T}} t' : \alpha$, that there exists p_1 such that $\Gamma' \vdash_{\mathcal{T}} p_1 : e$ and that there exists p_2 such that $\Gamma' \vdash_{\mathcal{T}} p_2 : e'$ where $\llbracket \Gamma' \rrbracket_{\text{ctx}} = \Gamma$, $\llbracket t' \rrbracket_{\text{ter}} = t$, $\llbracket e \rrbracket_{\text{ter}} = t \perp$, $\llbracket e' \rrbracket_{\text{ter}} = t \top$, $\llbracket \alpha \rrbracket_{\text{ty}} = \Pi_{x:o} o$ and $\llbracket \mathcal{T} \rrbracket_{\text{sig}} = T$. From the definition of the translation function we further know that $\alpha = \forall_{x:\mathbb{P}} \mathbb{P}$, that $e = t' \perp$ and that $e' = t' \top$. As stated earlier for this case we assume axiomatisations of a constant $\vdash_{\mathcal{T}} \text{lem} : \forall_{e:\mathbb{P}} e \vee (\neg e)$, the empty type $\perp$, the singleton type $\top$, the co-product $\vee$ and negation $\neg$. We need to prove that there exists p such that $\Gamma'' \vdash_{\mathcal{T}} p : e''$, where $\llbracket \Gamma'' \rrbracket_{\text{ctx}} = \Gamma, x : o$, $\llbracket e'' \rrbracket_{\text{ter}} = t \ x$. We choose $\Gamma'' = \Gamma', x : \mathbb{P}$, $e'' = t' \ x$. Then $\llbracket \Gamma'' \rrbracket_{\text{ctx}}$ and $\llbracket e'' \rrbracket_{\text{ter}}$ compute as desired. To acquire p , we use the induction principle associated with $\vee$ on **lem** x with e and e' , which yields the desired result. $\square$

We do not prove the cases regarding $\top$, $\perp$, $\wedge$, $\vee$, $\neg$ and $\exists$, since they depend on some design decisions when axiomatising their counterparts in DTT. For commonly used definitions, however, these easily follow in the same style as the other cases.

From Theorem 2, we easily derive soundness:

Theorem 3 (Soundness). *Let $\mathcal{T}$ be a translatable DTT3 signature, let Γ be a translatable DTT3 context, let e be a translatable DTT3 expression such that $\Gamma \vdash_{\mathcal{T}} e : \mathbb{P}$. If $\llbracket \Gamma \rrbracket_{\text{ctx}} \vdash_{\llbracket \mathcal{T} \rrbracket_{\text{sig}}}^{\text{DHOL}} \llbracket e \rrbracket_{\text{ter}}$, then there exists p such that $\Gamma \vdash_{\mathcal{T}}^{\text{DTT3}} p : e$.*

Proof. Consequence of the first non-header row of Table 2 and injectivity together with subject reduction. $\square$

6 Related Work

Our work extends the recent research on DHOL. Rothgang et al. [30] introduced the original monomorphic DHOL, including a sound and complete translation to monomorphic HOL. DHOL is reminiscent of the formalisms underlying the PVS [23] and Nuprl [12] proof assistants. In a follow-up unpublished manuscript, Ranalter et al. [26] generalised the logic and the translation to support rank 1 polymorphism. This generalised logic is the target of our own translation.

Rothgang and Rabe [29] also extended monomorphic DHOL with subtyping and adapted the translation to HOL accordingly. Moreover, Ranalter et al. [27] described an extension of monomorphic DHOL with Hilbert's choice operator and adapted the translation to HOL. On the theorem proving front, Niederhauser et al. [21] designed a sound and complete tableau style proof calculus for monomorphic DHOL and implemented it in the Lash prover. Finally, Ranalter et al. [28] extended the TPTP (Thousands of Problems for Theorem Provers) syntax and infrastructure with support for polymorphic DHOL.

Apart from the aforementioned translations from variants of DHOL to HOL, translations from DTTs are also relevant to our work. Jacobs and Melham [16]

encoded Martin-Löf type theory into polymorphic HOL. In the context of hammers, Czajka and Kaliszyk [13] and Qian et al. [25] developed pragmatic translations from DTT to FOL and HOL, respectively. Oury [22], Winterhalter et al. [33] and Vaishnav [31] all translated extensional DTTs to intensional DTTs. These three approaches therefore take an inverse direction to our work, which translates from an intensional logic to an extensional logic.

7 Conclusion

We presented a translation from a fragment of DTT to polymorphic DHOL. The translation is sound and lightweight. The translatable fragment constitutes a large part of DTT as used in practice. The translation could be used to integrate DHOL based provers into proof assistants based on DTT as part of a hammer.

There are several avenues for future work. First, we should try to prove completeness. Second, we want to extend the translation to support more advanced features of DTT (e.g. inductive definitions and quotient types). Third, we would like to formalise the soundness argument using a proof assistant. Finally, to reach users we should implement the translation in a hammer system.

Acknowledgements. We thank Massin Guerdi for invaluable discussions on the definitions and theorems. We thank an anonymous reviewer for pointing out how (D)HOL can be represented as a PTS. We thank Mark Summerfield and the anonymous reviewers for helpful textual suggestions.

Blanchette’s research was co-funded by the European Union (ERC, Nekoka, 101083038). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

1. Andrews, P.: An Introduction to Mathematical Logic and Type Theory, Applied Logic Series, vol. 27. Springer Dordrecht (2002)
2. Angiuli, C., Gratzer, D.: Principles of Dependent Type Theory. Cambridge University Press, to appear
3. Asperti, A., Ricciotti, W., Coen, C.S., Tassi, E.: The Matita interactive theorem prover. In: Bjørner, N.S., Sofronie-Stokkermans, V. (eds.) CADE-23. LNCS, vol. 6803, pp. 64–69. Springer (2011). https://doi.org/10.1007/978-3-642-22438-6_7
4. Barendregt, H., Dekkers, W., Statman, R.: Lambda Calculus with Types. Perspectives in Logic, Cambridge University Press (2013)
5. Bertot, Y., Castéran, P.: Interactive Theorem Proving and Program Development – Coq’Art: The Calculus of Inductive Constructions. Texts in Theoretical Computer Science. An EATCS Series, Springer (2004). <https://doi.org/10.1007/978-3-662-07964-5>

6. Blanchette, J.C., Böhme, S., Popescu, A., Smallbone, N.: Encoding monomorphic and polymorphic types. *Log. Methods Comput. Sci.* **12**(4) (2016). [https://doi.org/10.2168/LMCS-12\(4:13\)2016](https://doi.org/10.2168/LMCS-12(4:13)2016)
7. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. *J. Formaliz. Reason.* **9**(1), 101–148 (2016). <https://doi.org/10.6092/ISSN.1972-5787/4593>
8. Brown, C.E.: Satallax: An automatic higher-order prover. In: Gramlich, B., Miller, D., Sattler, U. (eds.) *IJCAR 2012*. LNCS, vol. 7364, pp. 111–117. Springer (2012). https://doi.org/10.1007/978-3-642-31365-3_11
9. Carneiro, M.: The Type Theory of Lean. Master’s thesis, Carnegie Mellon University (2019), <https://github.com/digama0/lean-type-theory/releases/tag/v1.0>
10. Church, A.: A formulation of the simple theory of types. *J. Symb. Log.* **5**(2), 56–68 (1940). <https://doi.org/10.2307/2266170>
11. Cohen, C.: Pragmatic quotient types in Coq. In: Blazy, S., Paulin-Mohring, C., Pichardie, D. (eds.) *Interactive Theorem Proving - 4th International Conference, ITP 2013, Rennes, France, July 22–26, 2013*. Proceedings. Lecture Notes in Computer Science, vol. 7998, pp. 213–228. Springer (2013). https://doi.org/10.1007/978-3-642-39634-2_17
12. Constable, R.L., Allen, S.F., Bromley, M., Cleaveland, R., Cremer, J.F., Harper, R., Howe, D.J., Knoblock, T.B., Mendler, N.P., Panangaden, P., Sasaki, J.T., Smith, S.F.: *Implementing Mathematics with the Nuprl Proof Development System*. Prentice Hall (1986), <http://dl.acm.org/citation.cfm?id=10510>
13. Czakajka, L., Kaliszyk, C.: Hammer for Coq: Automation for dependent type theory. *J. Autom. Reason.* **61**(1-4), 423–453 (2018). <https://doi.org/10.1007/S10817-018-9458-4>
14. Geuvers, J.: The calculus of constructions and higher order logic, pp. 139–191. *Cahiers du centre de logique*, Katholieke Universiteit Leuven, Belgium (1994)
15. Gordon, M.J.C.: Introduction to the HOL system. In: Archer, M., Joyce, J.J., Levitt, K.N., Windley, P.J. (eds.) *1991 International Workshop on the HOL Theorem Proving System and Its Applications*. pp. 2–3. IEEE Computer Society (1991)
16. Jacobs, B., Melham, T.F.: Translating dependent type theory into higher order logic. In: Bezem, M., Groote, J.F. (eds.) *TLCA ’93*. LNCS, vol. 664, pp. 209–229. Springer (1993). <https://doi.org/10.1007/BFB0037108>
17. Maio, L., Bentkamp, A., Blanchette, J., Turret, S.: From dependent type theory to dependently typed higher order logic (technical report) (2026), <https://nekoka-project.github.io/pubs/dtt-to-dhol-report.pdf>
18. Meng, J., Paulson, L.C.: Translating higher-order clauses to first-order clauses. *J. Autom. Reason.* **40**(1), 35–60 (2008). <https://doi.org/10.1007/S10817-007-9085-Y>
19. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Platzzer, A., Sutcliffe, G. (eds.) *CADE-28*. LNCS, vol. 12699, pp. 625–635. Springer (2021). https://doi.org/10.1007/978-3-030-79876-5_37
20. Nederpelt, R., Geuvers, H.: *Type Theory and Formal Proof: An Introduction*. Cambridge University Press (2014)
21. Niederhauser, J., Brown, C.E., Kaliszyk, C.: Tableaux for automated reasoning in dependently-typed higher-order logic. In: Benz Müller, C., Heule, M.J.H., Schmidt, R.A. (eds.) *IJCAR 2024, Part I*. LNCS, vol. 14739, pp. 86–104. Springer (2024). https://doi.org/10.1007/978-3-031-63498-7_6
22. Oury, N.: Extensionality in the calculus of constructions. In: Hurd, J., Melham, T.F. (eds.) *TPHOLs 2005*. LNCS, vol. 3603, pp. 278–293. Springer (2005). https://doi.org/10.1007/11541868_18

23. Owre, S., Rajan, S., Rushby, J.M., Shankar, N., Srivas, M.K.: PVS: Combining specification, proof checking, and model checking. In: Alur, R., Henzinger, T.A. (eds.) CAV '96. LNCS, vol. 1102, pp. 411–414. Springer (1996). https://doi.org/10.1007/3-540-61474-5_91
24. Pfenning, F., Paulin-Mohring, C.: Inductively defined types in the calculus of constructions. In: Main, M.G., Melton, A., Mislove, M.W., Schmidt, D.A. (eds.) MFPS 1989. LNCS, vol. 442, pp. 209–228. Springer (1989). <https://doi.org/10.1007/BFB0040259>
25. Qian, Y., Clune, J., Barrett, C.W., Avigad, J.: Lean-auto: An interface between Lean 4 and automated theorem provers. CoRR **abs/2505.14929** (2025). <https://doi.org/10.48550/ARXIV.2505.14929>
26. Ranalter, D., Rabe, F., Kaliszyk, C.: Polymorphic theorem proving for DHOL (2025), unpublished manuscript
27. Ranalter, D., Brown, C., Kaliszyk, C.: Experiments with choice in dependently-typed higher-order logic. In: Bjørner, N., Heule, M., Voronkov, A. (eds.) Proceedings of 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning. EPIC Series in Computing, vol. 100, pp. 311–320. EasyChair (2024). <https://doi.org/10.29007/2v8h>, [/publications/paper/IRJ1](https://publications/paper/IRJ1)
28. Ranalter, D., Kaliszyk, C., Rabe, F., Sutcliffe, G.: The dependently typed higher-order form for the TPTP world. In: Thiemann, R., Weidenbach, C. (eds.) FroCoS 2025. LNCS, vol. 15979, pp. 287–305. Springer (2025). https://doi.org/10.1007/978-3-032-04167-8_16
29. Rothgang, C., Rabe, F.: Subtyping in dependently-typed higher-order logic. In: Thiemann, R., Weidenbach, C. (eds.) FroCoS 2025. LNCS, vol. 15979, pp. 98–114. Springer (2025). https://doi.org/10.1007/978-3-032-04167-8_6
30. Rothgang, C., Rabe, F., Benz Müller, C.: Dependently typed higher-order logic. ACM Trans. Comput. Logic **27**(1) (Dec 2025). <https://doi.org/10.1145/3771725>, <https://doi.org/10.1145/3771725>
31. Vaishnav, R.: Lean4less: Eliminating definitional equalities from lean via an extensional-to-intensional translation. In: Liu, Z., Saoud, A., Wehrheim, H. (eds.) Theoretical Aspects of Computing – ICTAC 2025. pp. 202–219. Springer Nature Switzerland, Cham (2026)
32. Werner, B.: Sets in types, types in sets. In: Abadi, M., Ito, T. (eds.) Theoretical Aspects of Computer Software. pp. 530–546. Springer Berlin Heidelberg, Berlin, Heidelberg (1997)
33. Winterhalter, T., Sozeau, M., Tabareau, N.: Eliminating reflection from type theory. In: Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs. p. 91–103. CPP 2019, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3293880.3294095>, <https://doi.org/10.1145/3293880.3294095>